Docket UI User Guide

Two people's journeys through Docket: a policy servicer in the workbench, and an assessor in the explorer. At every step you see what you decide, what Docket gives you back, the control to use boxed in red, and the result it produces.

Screenshots come from the app on `main`, with the trail compiled at snapshot `a2dc6df`. Every path is under `src/Uinsure.Policies.Web` or `tests/Uinsure.Policies.Web.Tests`.

The big ideas

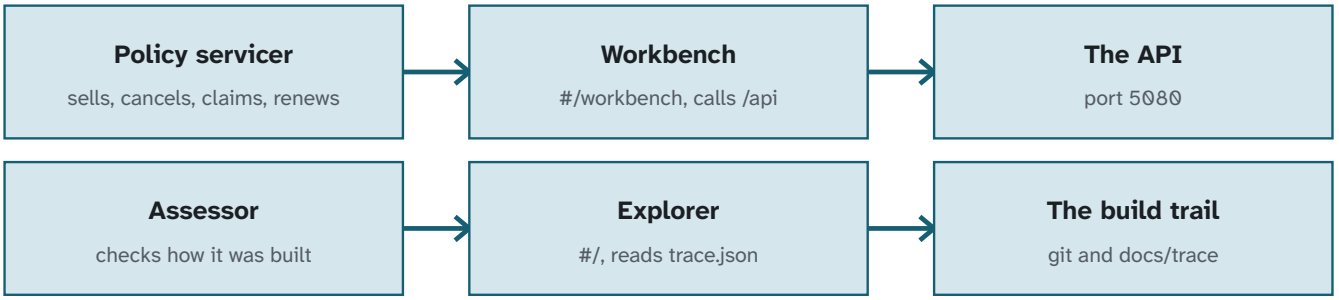
1. **Two people.** A policy servicer drives the API from the workbench; an assessor checks how it was built from the explorer. Pick your journey and follow it.
2. **Decide, then see.** Every step names what you decide and what Docket gives you back, so the screen is never a surprise.
3. **Red boxes.** The control a step drives from has a red box and a "Step n" label. The shot shows the result the step produces.
4. **Evidence.** Every rule links to its requirement, its tests, its commits and its verdicts, so one click replaces a search of the git log.
5. **Honest.** A missing value is drawn as missing, an answer is shown with the status the page received, and nothing is invented.
6. **Contracts.** Screens were agreed as static HTML first, and the app is tested against them: unit, end to end, axe, parity and visual regression.
7. **Run it yourself.** Every test collection can be run by hand: Vitest, Playwright and Postman.

Start here

Pick your journey, then follow its steps in order.

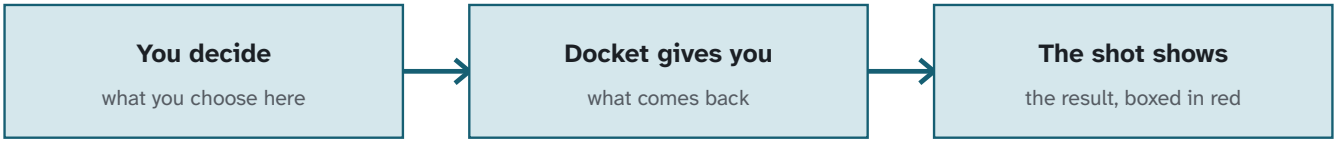
Docket is one app with two halves. The workbench sends real requests to the API and shows what came back. The explorer reads the build trail and shows how each rule was proved. Each half has its own journey below, and each step in a journey follows the same shape.

▼ In depth



Two people, two halves of one app

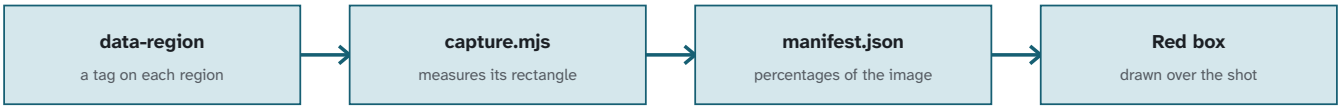
	Policy servicer	Assessor
Works in	The workbench, `#/workbench`	The explorer, `#/`
Reads	The live API	`trace.json`, compiled from git and the trail files
Needs running	The API, and the app with `/api` pointed at it	The app only
Journey	SJ1 to SJ9	AJ1 to AJ8



Every step has the same shape

How to read a screenshot

A red box is drawn round the control or region a step drives from, with a red "Step n" label beside it. The sub-step list under the shot says what to do at "Step n". A shot with no box frames a result: the response, the end of a page, a plan picture. Box positions are measured from each region's `data-region` tag by `capture.mjs`, never placed by eye.



Where a red box comes from

> **Two kinds of Step**
> Explorer screens print their own rung labels in black, such as *Step 1 Read the answer*. Those belong to the screen. The red *Step n* labels belong to this guide. Where a screen prints its own labels, the step text says which one it means. The workbench forms print numbered circles for their rungs; they are not boxes or labels either.

> **Why**
> Choice: Box each control from the app's own region tags. Alternative: Place boxes by hand. Reason: A hand-placed box drifts when the layout changes. A measured box cannot lie about where a tagged region is.

> **Untagged regions**
> Four regions carry no `data-region` tag: the header and theme toggle, the cancel confirmation dialog, the requests drawer, and a problem banner inside a screen. Their boxes are measured from the element found by its accessible name, and the manifest records the nearest tagged ancestor. Nothing is placed by hand.

Check your understanding

Question	Answer
Which journey is mine?	Servicers follow SJ1 to SJ9 in the workbench. Assessors follow AJ1 to AJ8 in the explorer.
What is the red "Step n" label?	This guide's label for the control you use at that sub-step. It is not the screen's own rung label.
Where do the boxes come from?	The `data-region` tag of the region, measured by `capture.mjs`.

Journey 1: the policy servicer, in the workbench

From opening the workbench to a cancelled policy, a claim, a refused renewal and the requests behind them.

You sell a policy, read it, quote a cancellation and cancel, record a claim, try a renewal, and read the requests you sent. At each step the workbench shows the status the API answered, never one it did not receive. Forms validate as you go, the refund is shown before you commit, every call is logged with a curl, and the rule behind each refusal is one click away.

1 SJ1 · Open the workbench

> **You decide** Whether to serve a static build or the dev server, and which port your API is on.

> **Docket gives you** The workbench home with its status line. It says nothing has been requested until you press Open policy. Both modes proxy `/api` to the API.

Mode	Command, from the repository root unless stated	App port	Use when
Static build, preview	`run build` then `run preview -- --port 4170`	4170	A demo; the screenshots in this guide came from a static build
Dev server	`node scripts/dev.mjs --port 4170 --strictPort`, from `src/Uinsure.Policies.Web`	4170	Changing Docket; it compiles the trail on start
Visual Studio	Open `Uinsure.Policies.slnx` and start the "API and web" profile	4170	Starts the API (http profile, 5080) and the dev server together

> **Shell**
> The commands in this guide are for Git Bash or any POSIX shell. In PowerShell, set the port first with `$env:DOCKET_API_PORT = "5080"` instead of the `DOCKET_API_PORT=5080` prefix, and use `Invoke-WebRequest` where this guide uses `curl`.

1. Install the packages once.

```
npm --prefix src/Uinsure.Policies.Web ci
```

2. Compile the trail into `trace.json` (the explorer's data), then build the static app.

```
npm --prefix src/Uinsure.Policies.Web run trace
npm --prefix src/Uinsure.Policies.Web run build
```

You should see: Both finish with exit `0`; the build took 1.3 s.

```
> trace
> node compiler/index.ts --ref HEAD
…
…
dist/assets/index-DvJq6KFN.css                                    4.08 kB │ gzip:
0.88 kB
dist/assets/kitchen-sink-BrcHVApU.css                             4.77 kB │ gzip:
1.11 kB
dist/assets/VerdictCard-BdZWhHC7.css                            136.79 kB │ gzip:
16.30 kB
dist/assets/kitchen-sink-CRk5hoo8.js                            115.98 kB │ gzip:
28.68 kB
dist/assets/index-B4_AN9ly.js                                   191.66 kB │ gzip:
56.12 kB
dist/assets/VerdictCard-8Tz1koC-.js                             374.15 kB │ gzip:
111.06 kB

✓ built in 643ms
```

3. Start the API on its http profile in one terminal.

```
dotnet run --project src/Uinsure.Policies.Api --launch-profile http
```

You should see: The API listens on `5080`.

```
Using launch settings from src\Uinsure.Policies.Api\Properties\launchSettings.json...
…
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: <repo>\src\Uinsure.Policies.Api
```

4. In a second terminal, preview the app with `/api` proxied to that port.

```
DOCKET_API_PORT=5080 npm --prefix src/Uinsure.Policies.Web run preview -- --port 4170
```

You should see: The preview serves the app on `4170`.

```
> preview
> vite preview --port 4170 --strictPort

  ➜  Local:   http://localhost:4170/
  ➜  Network: use --host to expose
```

5. Check that the app, the API and the proxy each answer.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:4170/
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:5080/health/live
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:4170/api/health/live
```

You should see: HTTP `200` from each, from this run.

```
200
200
200
```

Open `http://localhost:4170/#/workbench`. The shot shows what you should see before you send anything.



Workbench home: open a policy by reference, with the request status under it. `#/workbench` Requirement tags in view: **WB-2** **BUSY-1**

Step 1: The **Policy reference** box. Type the reference of a policy that exists. You have none yet, so leave it empty and sell one in SJ2.

Step 2: The **Open policy** button. It sends one `GET /policies/<reference>`, and shows Opening… and is disabled until the API answers.

You should see: Request status says *Idle. No request sent.* The workbench never shows a result it did not receive.

`src/Uinsure.Policies.Web/app/workbench/home/WorkbenchHome.tsx:32`

```
export function WorkbenchHome() {
  const navigate = useNavigate();
  const [reference, setReference] = useState('');
  const [opening, setOpening] = useState<Opening>({ kind: 'idle' });
  const busy = opening.kind === 'busy';
  const recent = readRecent(browserDeps().storage);
```

`src/Uinsure.Policies.Web/vite.config.ts:19`

```
const proxy = {
  '/api': {
    target: `http://localhost:${process.env.DOCKET_API_PORT ?? 5080}`,
    rewrite: (path: string) => path.replace(/^\/api/, ''),
  },
};
```

`src/Uinsure.Policies.Web/Docket.Web.esproj:4`

```
    <StartupCommand>node scripts/dev.mjs --port 4170 --strictPort</StartupCommand>
    <JavaScriptTestRoot>..\..\tests\Uinsure.Policies.Web.Tests\</JavaScriptTestRoot>
    <JavaScriptTestFramework>Vitest</JavaScriptTestFramework>
```

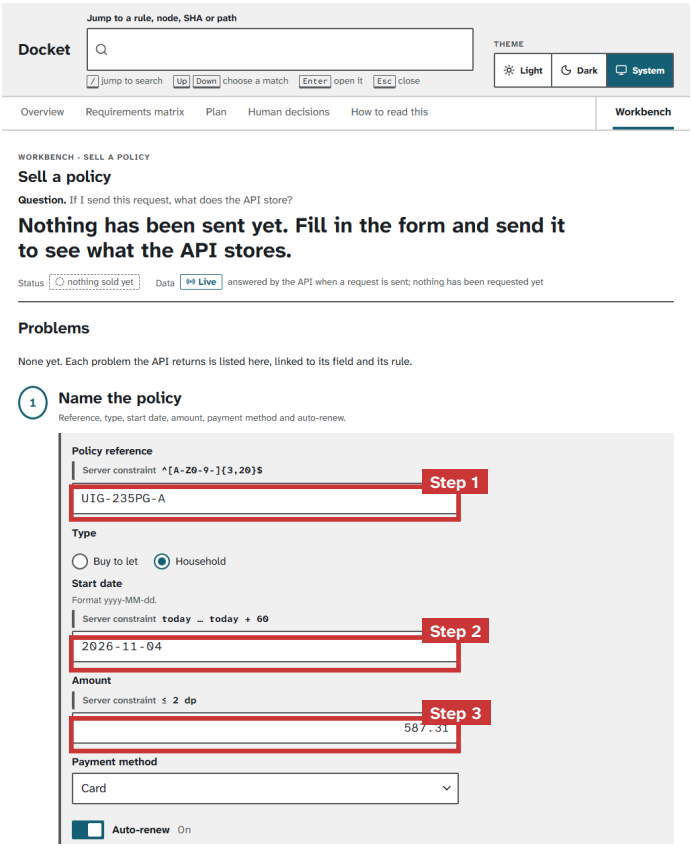
> **Why**
> Choice: Proxy `/api` to the API instead of calling it from the browser. Alternative: Call the API's origin directly. Reason: The page and the API then share one origin, so the browser makes no cross-origin call. The same shape serves the hosted Static Web App.

2 SJ2 · Sell a policy

> **You decide** The reference, the start date, the amount and payment method, one to three policyholders, and the property.

> **Docket gives you** Checks on each field as you type, then the API's answer: a new policy reference, or the rule that refused it.

Open `#/workbench/sell` from the home page's *Sell a new policy* link. The form has rungs: name the policy, add the policyholders, describe the property, then send. The grey *Server constraint* lines under a field are the API's own limits.

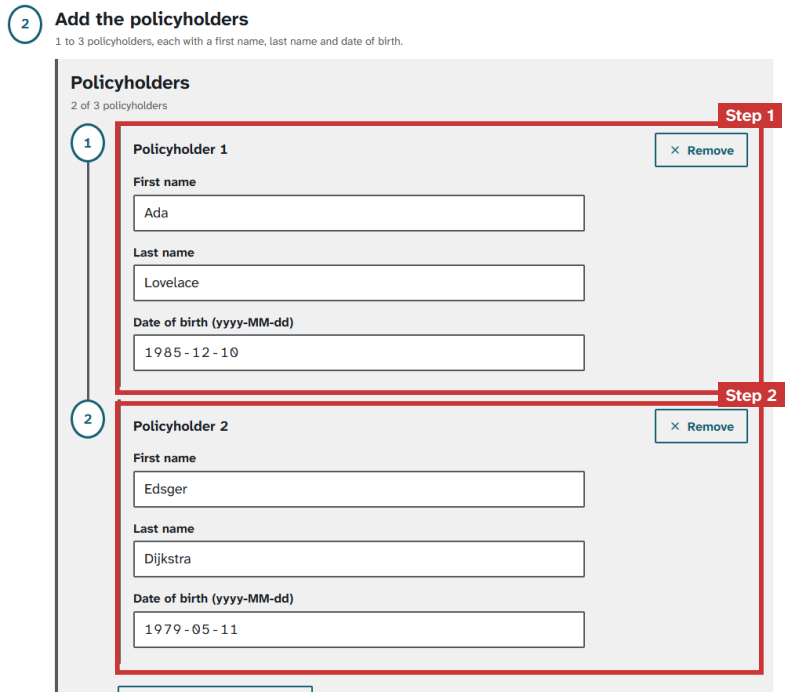


Sell form, filled in. `#/workbench/sell` Requirement tags in view: `WB-3` `WB-1`

Step 1: Policy reference: type a new one, 3 to 20 letters, digits or hyphens. It is trimmed and upper-cased as the API stores it (SELL-1).

Step 2: Start date: `yyyy-MM-dd`, from today to today + 60. No backdating (SELL-2).

Step 3: Amount: greater than zero, at most 2 decimal places (SELL-7). Choose Type, Payment method and Auto-renew above and below it.



Sell form, two policyholders. `#/workbench/sell` Requirement tags in view: **WB-1**

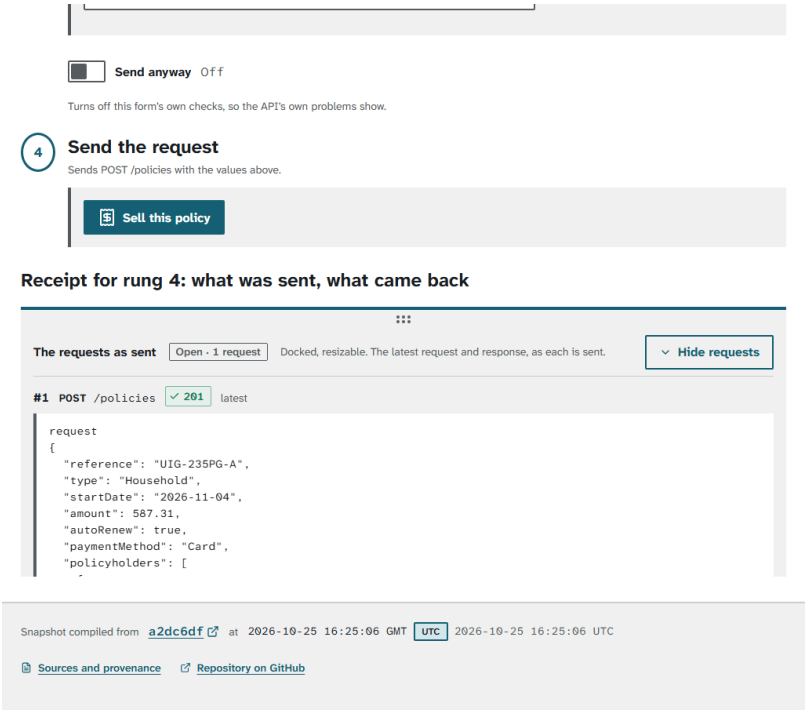
Step 1: Policyholder 1: first name, last name and date of birth. Each must be 16 or older on the start date (SELL-5).

Step 2: Policyholder 2: add up to three in all, remove with *Remove* (SELL-4).



Sell form, ready to send. `#/workbench/sell` Requirement tags in view: **BUSY-1** **WB-7**

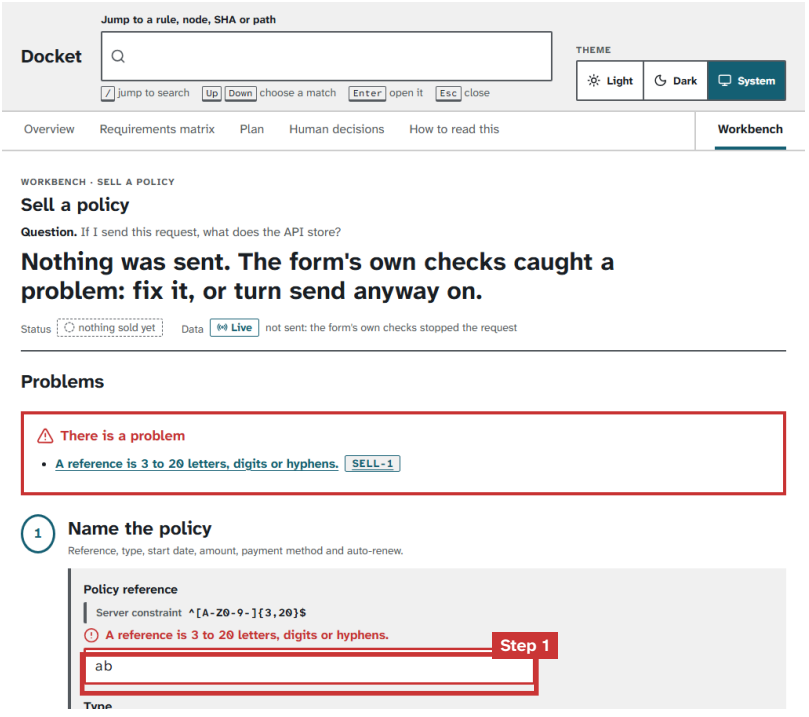
Step 1: Sell this policy: sends `POST /policies` with every value above. The button is busy and disabled while the request runs.



The result at the end of the form: the request as sent, and the API's 201. `#/workbench/sell` Requirement tags in view: **WB-1** **WB-6** **BUSY-1** **WB-7**

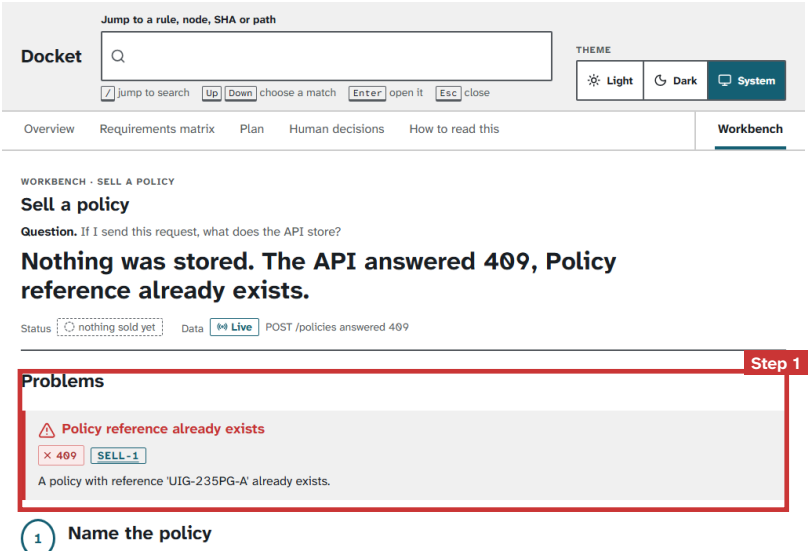
You should see: The receipt for rung 4 lists `POST /policies` with a green **201** and the request body as it went. Nothing is hidden or reformatted. Continue to SJ3 to open the policy.

When the sale is refused



Sell form, its own check stops a bad reference before anything is sent. `#/workbench/sell` Requirement tags in view: **WB-3** **WB-1**

Step 1: The **Policy reference** box holds `ab`. The form's own check stops it before anything is sent; the page says so, and no request appears in the log.



Sell form, the same reference again is refused (SELL-1, 409). `#/workbench/sell` Requirement tags in view: **WB-3** **WB-1**

Step 1: The **error summary**: the same reference sold twice. The API answers **409** with rule `SELL-1`, and the page repeats the API's own words.

`src/Uinsure.Policies.Web/app/workbench/sell/SellPolicy.tsx:130`

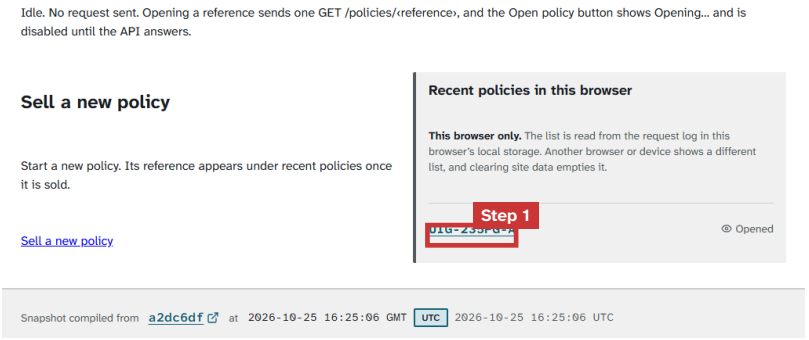
```
export function SellPolicy() {
  const [values, setValues] = useState<Values>(INITIAL);
  const [holders, setHolders] = useState<Holder[]>([blankHolder('holder-1')]);
  const [sendAnyway, setSendAnyway] = useState(false);
  const [outcome, setOutcome] = useState<Outcome>({ kind: 'idle' });
  const [attempt, setAttempt] = useState(0);
```

3 SJ3 · Read the policy

You decide Which policy to open: from the sell answer, the home page's recent list, or by typing its reference.

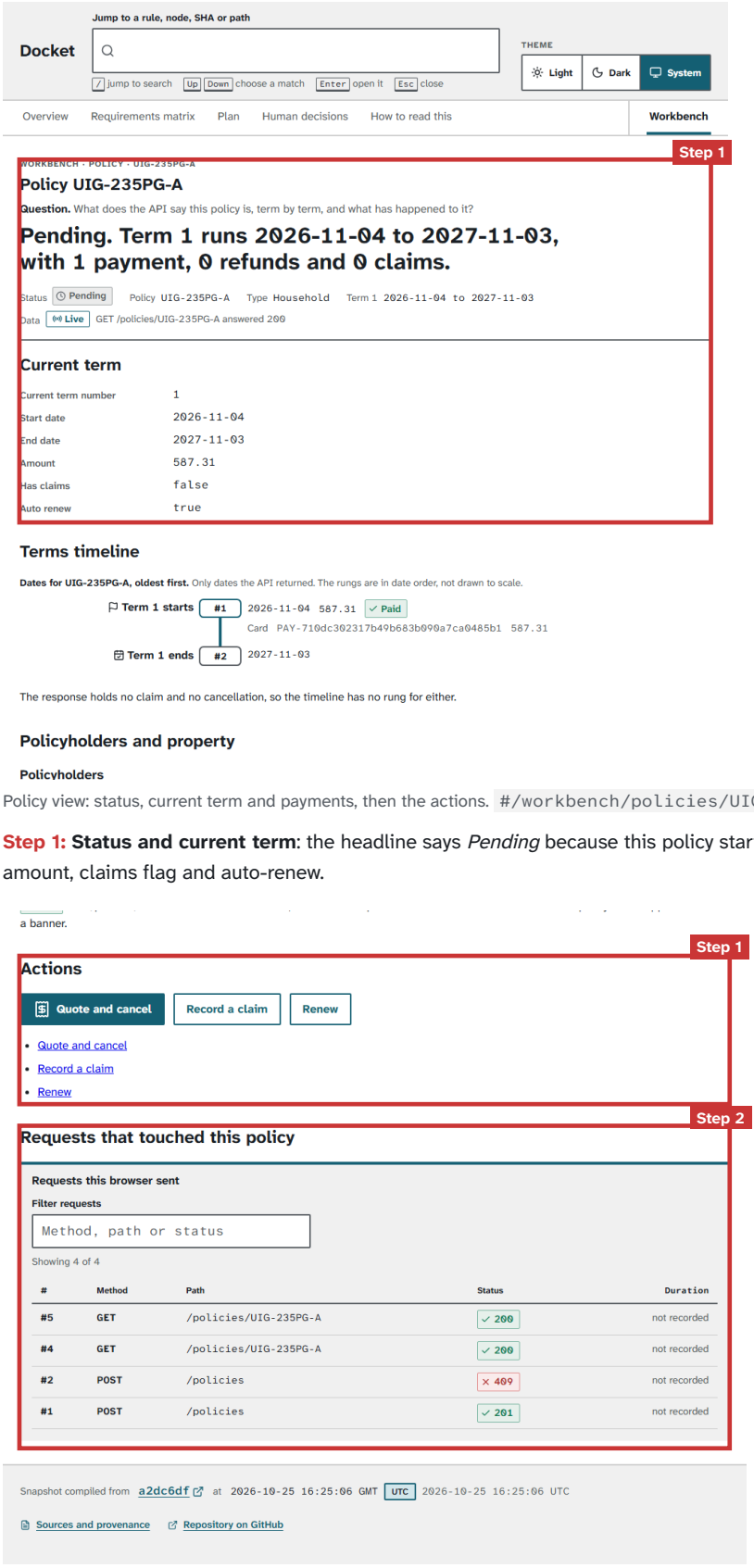
Docket gives you The policy as the API holds it: its status, current term, payments, holders and property, the actions open to you, and the requests that touched it.

Back on `#/workbench`, the browser remembers the references you have used and lists them under *Recent policies in this browser*. The list is local to this browser.



Workbench home, with a recent policy to open again. `#/workbench` Requirement tags in view: **BUSY-1** **WB-1** **WB-9**

Step 1: The recent policy **UIG-235PG-A**: choose it to open the policy page.



Policy view: status, current term and payments, then the actions. `#/workbench/policies/UIG-235PG-A` Requirement tags in view: **WB-2**

Step 1: Status and current term: the headline says *Pending* because this policy starts in the future. Below it the term's dates, amount, claims flag and auto-renew.

The same page, down to its last section. `#/workbench/policies/UIG-235PG-A` Requirement tags in view: **WB-4** **WB-5** **WB-7**

Step 1: Actions: Quote and cancel (SJ4), Record a claim (SJ5) and Renew (SJ6).

Step 2: Requests that touched this policy: the sell, the failed duplicate and each read, with the status the API sent.

src/Uinsure.Policies.Web/app/workbench/policy/PolicyView.tsx:323

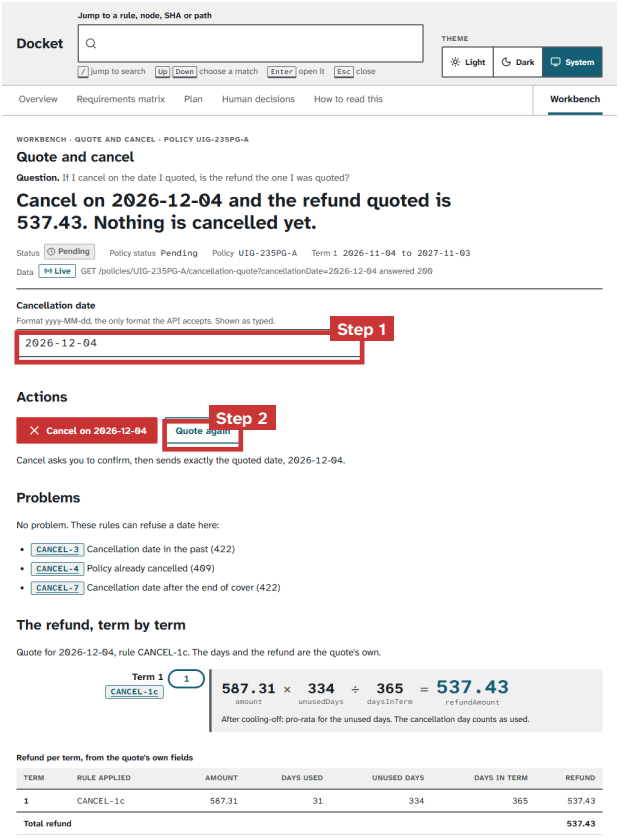
```
export function PolicyView() {
  const { reference = '' } = useParams();
  const cancelPath = cancelRoute(reference);
  const navigate = useNavigate();
  const [attempt, setAttempt] = useState(0);
  const key = `${reference}#${attempt}`;
```

4 SJ4 · Quote a cancellation, then cancel

> **You decide** The date to quote, and whether to cancel on that date once you have seen the refund.

> **Docket gives you** The refund for each term, worked as a sum, with the rule that decided it (CANCEL-1a, CANCEL-1b, CANCEL-1c or UNPAID-TERM). A confirmation dialog stands between you and the cancel.

From the policy's Actions press *Quote and cancel*. A quote is read-only: it saves nothing (COULD-1).

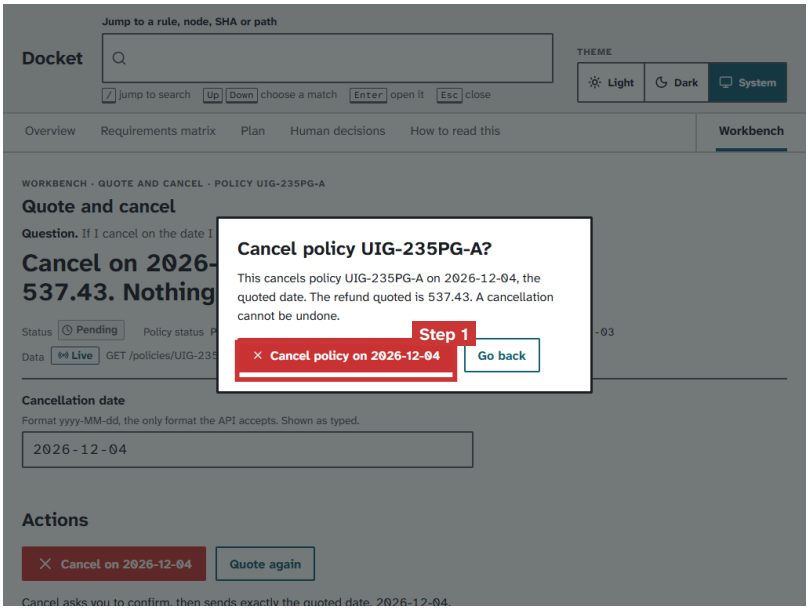


Cancel screen, the quote for the date: the refund per term and the rule behind it. `#/workbench/policies/UIG-235PG-A/cancel` Requirement tags in view: **WB-4** **BUSY-1** **WB-3**

Step 1: Cancellation date: type `2026-12-04`. Format `yyyy-MM-dd`, shown as you typed it.

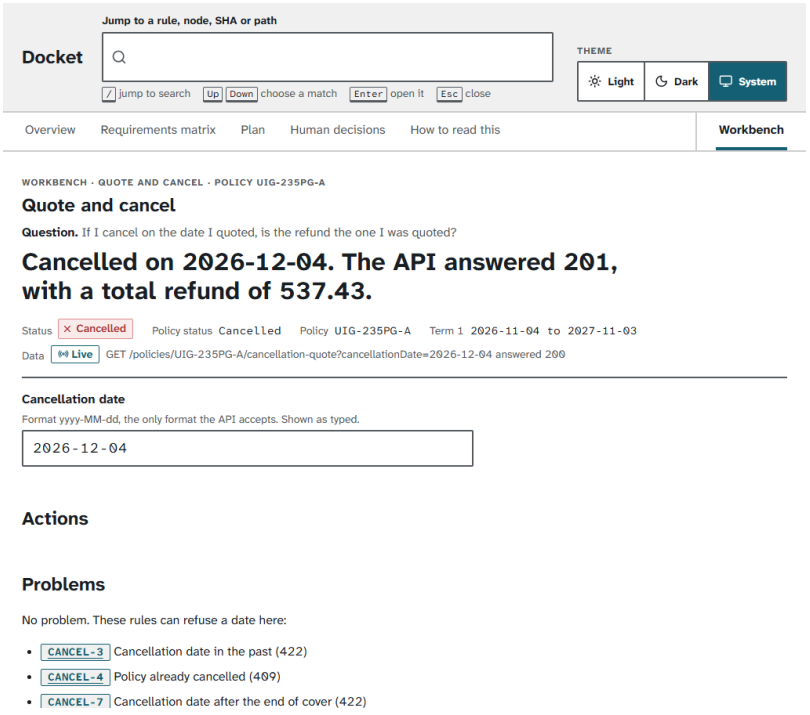
Step 2: Quote (*Quote this date*, then *Quote again*): the API answers 200. The headline reads *Cancel on 2026-12-04 and the refund quoted is 537.43. Nothing is cancelled yet.*

You should see: *The refund, term by term* shows 587.31 × 334 ÷ 365 = 537.43, rule `CANCEL-1c` (after the cooling-off period, pro-rata on the days unused). The Problems list names the rules that can refuse a date: CANCEL-3, CANCEL-4 and CANCEL-7.



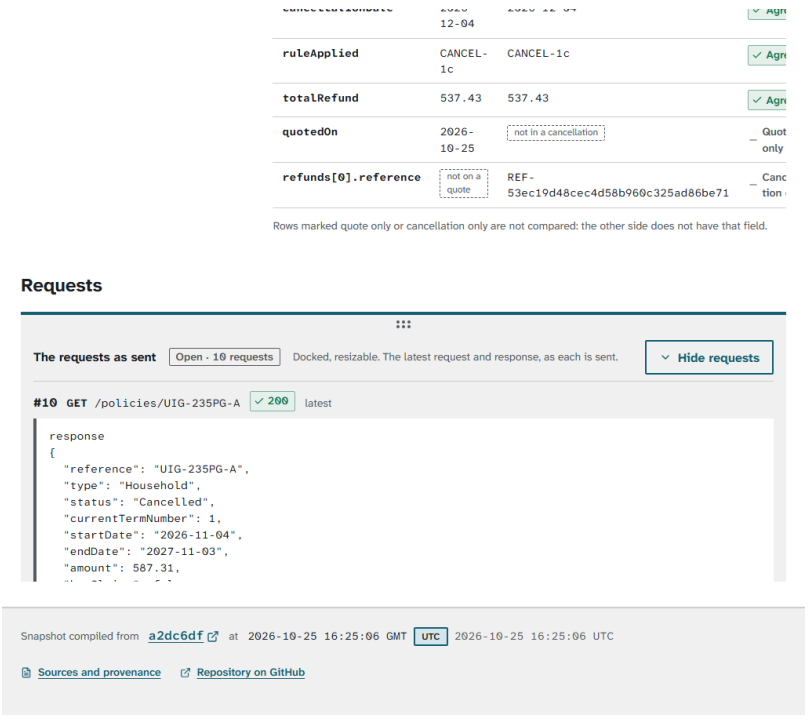
Cancel screen, the confirmation dialog. `#/workbench/policies/UIG-235PG-A/cancel` Requirement tags in view: WB-4 BUSY-1

Step 1: **Cancel policy on 2026-12-04** in the confirmation dialog. The dialog repeats the policy, the quoted date and the refund, and says a cancellation cannot be undone. *Go back* leaves the policy untouched.



Cancel screen, cancelled: the status and the refund. `#/workbench/policies/UIG-235PG-A/cancel` Requirement tags in view: WB-4 BUSY-1 WB-3

You should see: The headline reads *Cancelled on 2026-12-04. The API answered 201, with a total refund of 537.43.* The Status chip is Cancelled, re-read from the API.



The end of the page: the quote and the cancellation compared, field by field. `#/workbench/policies/UIG-235PG-A/cancel` Requirement tags in view: **WB-4** **WB-7**

You should see: `ruleApplied` and `totalRefund` agree between the quote and the cancellation. Rows marked quote-only or cancellation-only are not compared. The requests drawer shows the final `GET` that reported the policy as Cancelled.

`src/Uinsure.Policies.Web/app/workbench/cancel/CancelPolicy.tsx:71`

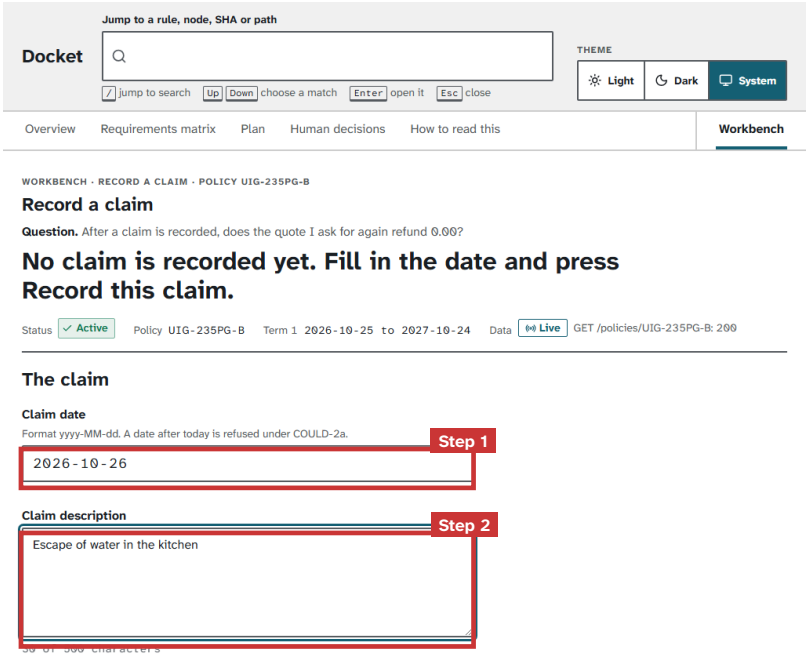
```
function CancelScreen({ reference }: { reference: string }) {
  const [policy, setPolicy] = useState<Policy | null>(null);
  // HONEST-1: true when the re-read after a cancel failed, so the status drawn is the one read before
the cancel.
  const [stale, setStale] = useState(false);
  const [date, setDate] = useState('');
  const [quoting, setQuoting] = useState<Step<CancellationQuote>>({ kind: 'idle' });
```

5 SJ5 · Record a claim

> **You decide** The date of the claim and a description.

> **Docket gives you** The refund quoted before the claim, the claim recorded, and the refund quoted again. A claim changes the refund (COULD-2).

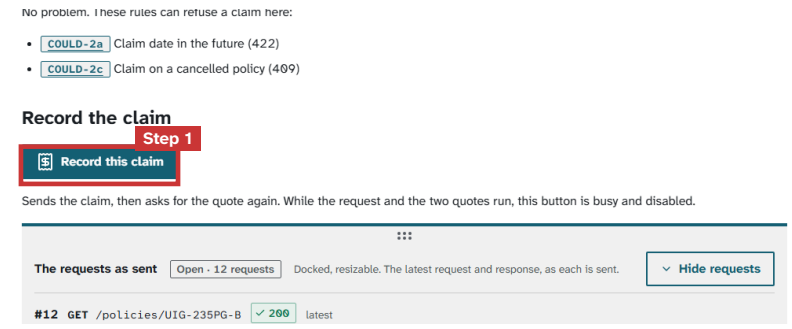
Open `#/workbench/policies/<reference>/claims/new` from the policy's Actions. The screen reads three rungs down: quote the refund, record the claim, quote again.



Record a claim, filled in. `#/workbench/policies/UIG-235PG-B/claims/new` Requirement tags in view: **WB-5**

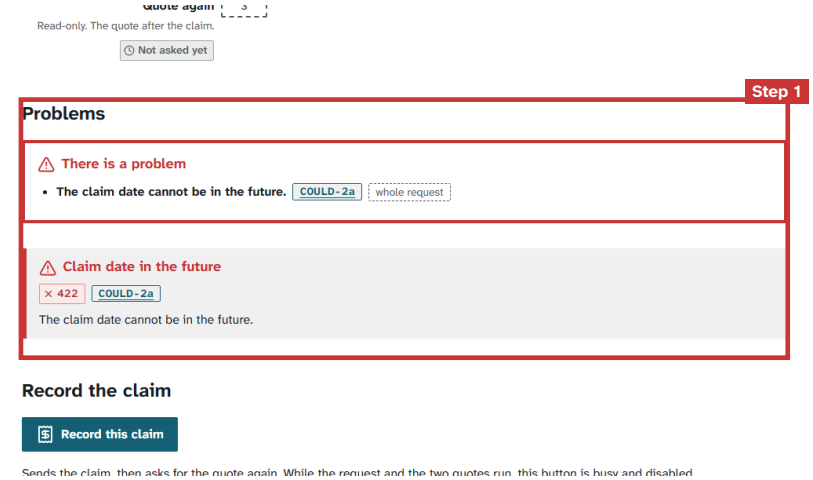
Step 1: Claim date: today, or an earlier date inside a term's cover (COULD-2a, COULD-2b).

Step 2: Claim description: up to 500 characters.



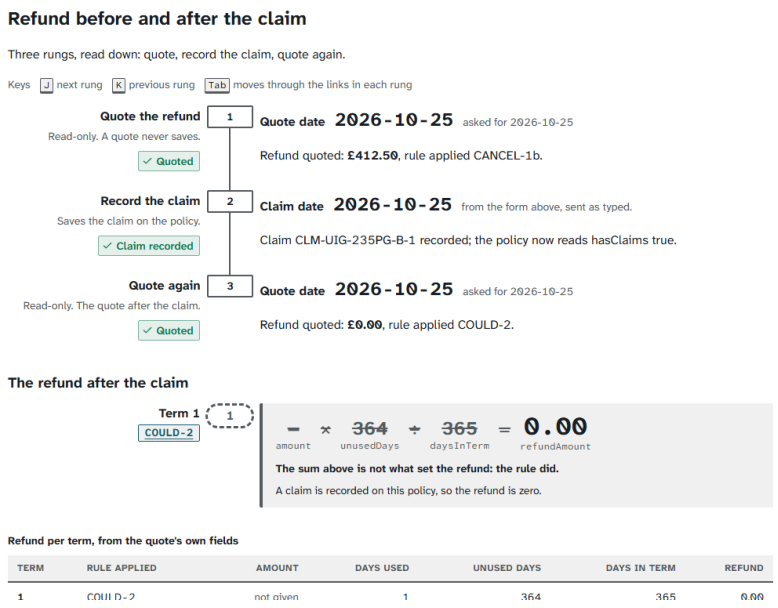
Record a claim, ready to send. `#/workbench/policies/UIG-235PG-B/claims/new` Requirement tags in view: **WB-3** **BUSY-1**

Step 1: Record this claim: sends the claim, then asks for the quote again. The button stays busy until the request and both quotes have run.



Record a claim, a future date is refused (COULD-2a, 422). `#/workbench/policies/UIG-235PG-B/claims/new` Requirement tags in view: **WB-5** **WB-3** **BUSY-1**

Step 1: The **problem list** after a claim dated in the future: the API answers **422** with `COULD-2a`. Nothing was recorded.



The claim recorded: the refund before and after. `#/workbench/policies/UIG-235PG-B/claims/new` Requirement tags in view: **WB-5**

You should see: Rung 1: refund quoted **£412.50**, rule `CANCEL-1b`. Rung 2: claim recorded; the policy now reads `hasClaims true`. Rung 3: refund quoted **£0.00**, rule `COULD-2`. The struck-through sum shows that the rule, not the arithmetic, set the refund.

`src/Uinsure.Policies.Web/app/workbench/claim/RecordClaim.tsx:85`

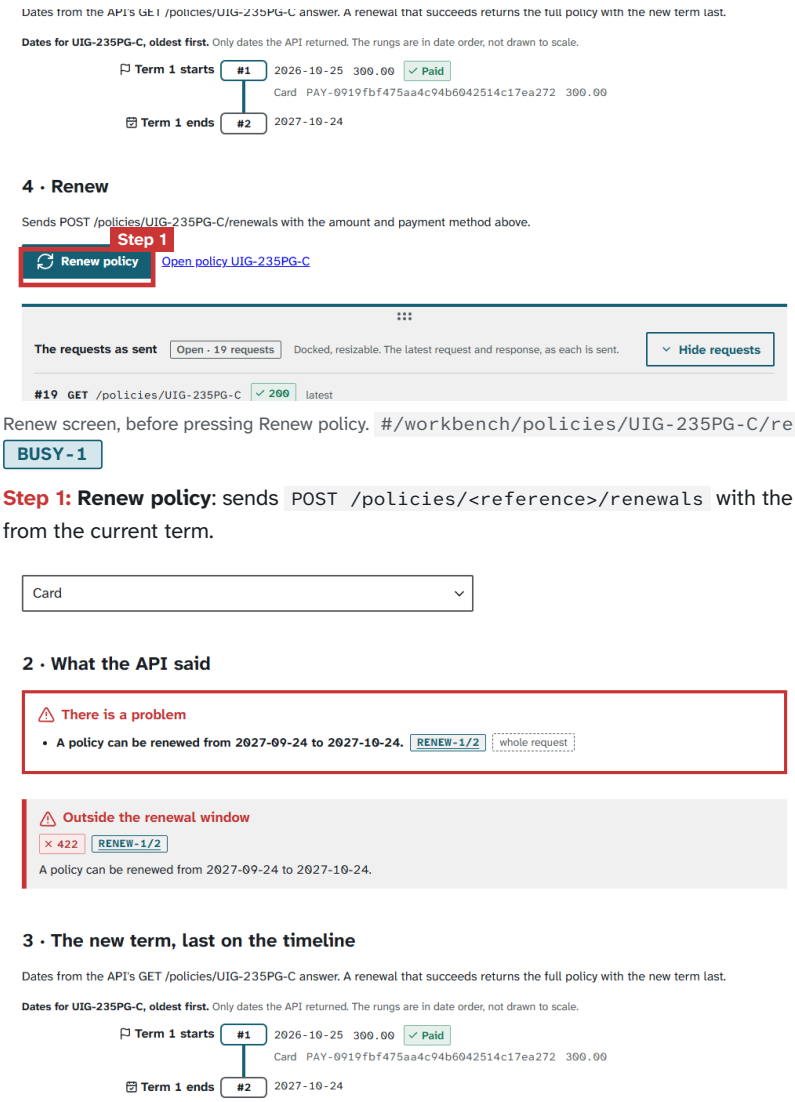
```
export function RecordClaim() {
  const { reference = '' } = useParams();
  const [policy, setPolicy] = useState<ApiAnswer<Policy> | null>(null);
  const [attempt, setAttempt] = useState(0);
  const [date, setDate] = useState(() => todayInLondon());
  const [description, setDescription] = useState('');
```

6 SJ6 · Renew

> **You decide** Whether to renew now, and the amount and payment method for the new term.

> **Docket gives you** The API's answer and, outside the renewal window, its refusal with the window's dates (RENEW-1, RENEW-2).

Renewal opens 30 days before the term ends and closes on its last day. A policy sold today runs for a year, so on the real clock a new policy cannot be renewed yet. This guide therefore shows the refusal. The success path is a gap: it needs a policy inside its window.



Renew screen, before pressing Renew policy. `#/workbench/policies/UIG-235PG-C/renew` Requirement tags in view: **WB-5** **WB-2** **BUSY-1**

Step 1: Renew policy: sends `POST /policies/<reference>/renewals` with the amount and payment method, which default from the current term.

You should see: *What the API said* shows a problem banner: *A policy can be renewed from 2027-09-24 to 2027-10-24*, with the chip `RENEW-1/2` and a **422**. The timeline below stays unchanged: nothing was renewed.

`src/Uinsure.Policies.Web/app/workbench/renew/RenewPolicy.tsx:62`

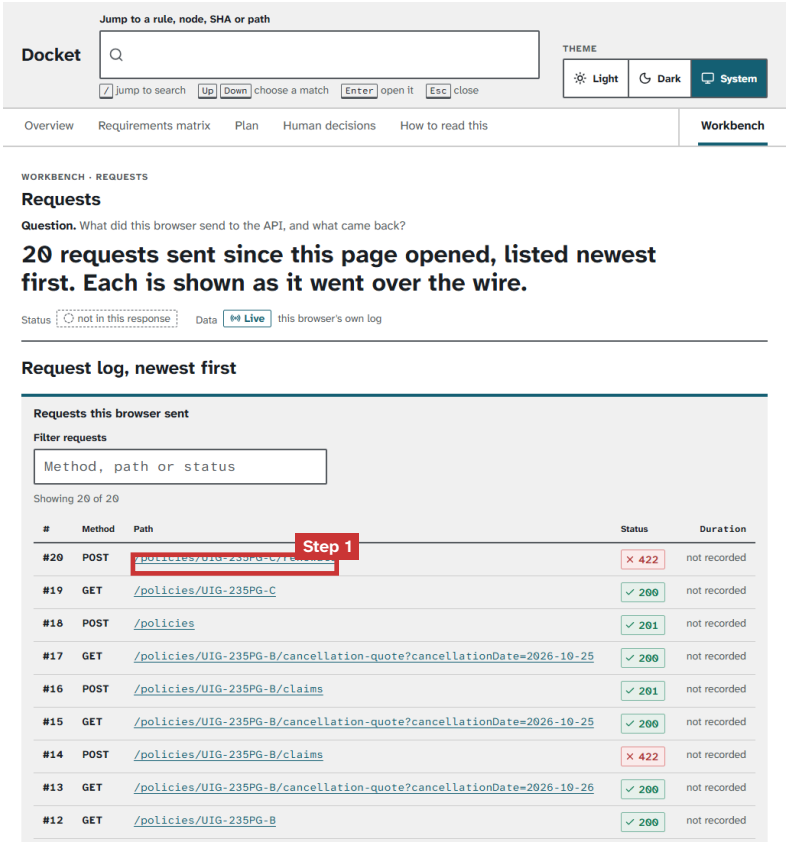
```
export function RenewPolicy() {
  const { reference = '' } = useParams();
  const [policy, setPolicy] = useState<ApiAnswer<Policy> | null>(null);
  const [attempt, setAttempt] = useState(0);
  const [amount, setAmount] = useState('');
  // WB-5: the term amount is only a starting value; it never replaces what the person typed first.
```

7 SJ7 · Inspect what you sent

> **You decide** Which exchange to open.

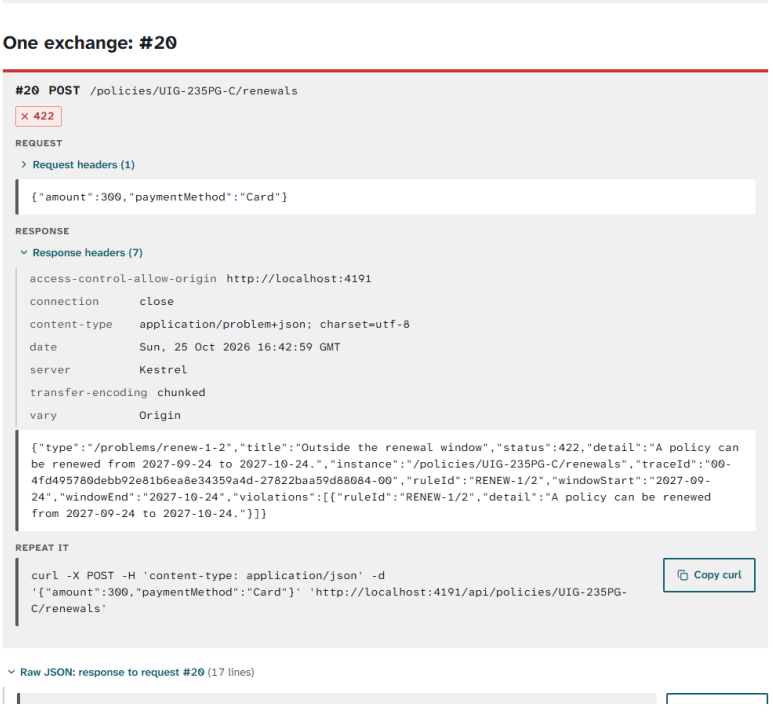
> **Docket gives you** The request log, newest first, and one exchange as it went over the wire, with a curl command to repeat it.

Every screen's requests drawer lists what this browser sent. `#/workbench/requests` lists all of them on one page.



Request log, newest first: the exchange to open. `#/workbench/requests` Requirement tags in view: **WB-7**

Step 1: A **path** in the log: `/policies/UIG-235PG-C/renewals`, the 422 from SJ6. Choose it to open the exchange.



One exchange open: request, response, status line and the curl to repeat it. `#/workbench/requests` Requirement tags in view: **WB-7**

You should see: The request body, the response headers and body (a problem JSON with `ruleId` `RENEW-1/2`), and under *Repeat it* a `curl` command with a *Copy curl* button. Run it in a terminal to get the same 422.

src/Uinsure.Policies.Web/app/workbench/requests/RequestLogPage.tsx:23

```
export function RequestLogPage() {
  const log = useRequestLog();
  const [picked, setPicked] = useState<number | null>(null);

  const numbered = log.map((entry, index) => ({ n: index + 1, entry }));
  const entries: RequestLogEntry[] = numbered
```

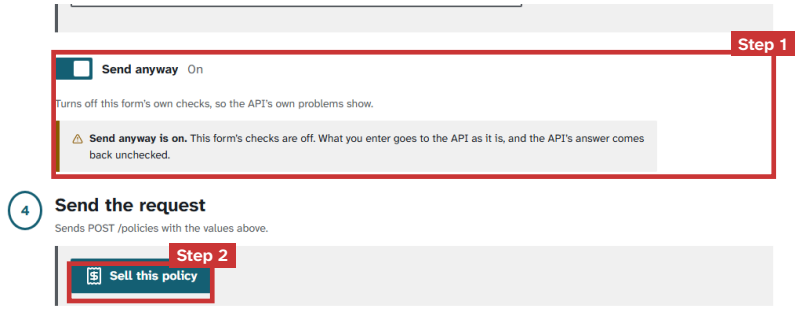
8 SJ8 · When it goes wrong

You decide Whether to fix the input, retry, or stop.

Docket gives you A problem banner with the status, the rule id and each violation. When the API is down the page says no request was completed. Not-found and empty states say so.

What happened	What you see	Status and rule	What to do
The form stops you	A problem list, and no request in the log	None sent	Fix the field
A business rule refuses	The API's own words, each violation with its rule chip	422, for example SELL-2	Change the input to meet the rule
A conflict	The API's detail and the rule chip	409, for example SELL-1	Use another reference
No such policy	The API's not-found answer	404, GET-1	Check the reference
The API is down	*The API did not answer*, *No request was completed*	No response	Start the API, then Try again
A page that does not exist	*Not in this snapshot*, nothing guessed	None	Use the link home

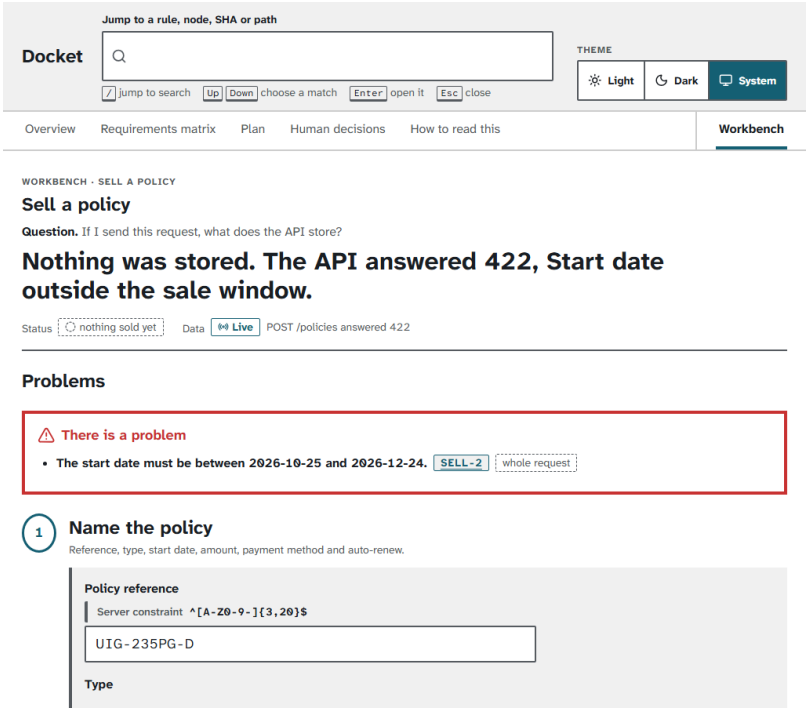
A 422: the rule refuses



Sell form, Send anyway on, ready to send a backdated start. `#/workbench/sell` Requirement tags in view: WB-6 BUSY-1

Step 1: **Send anyway** switch, On: it turns off the form's own checks, so you can see the API's answer to a bad request.

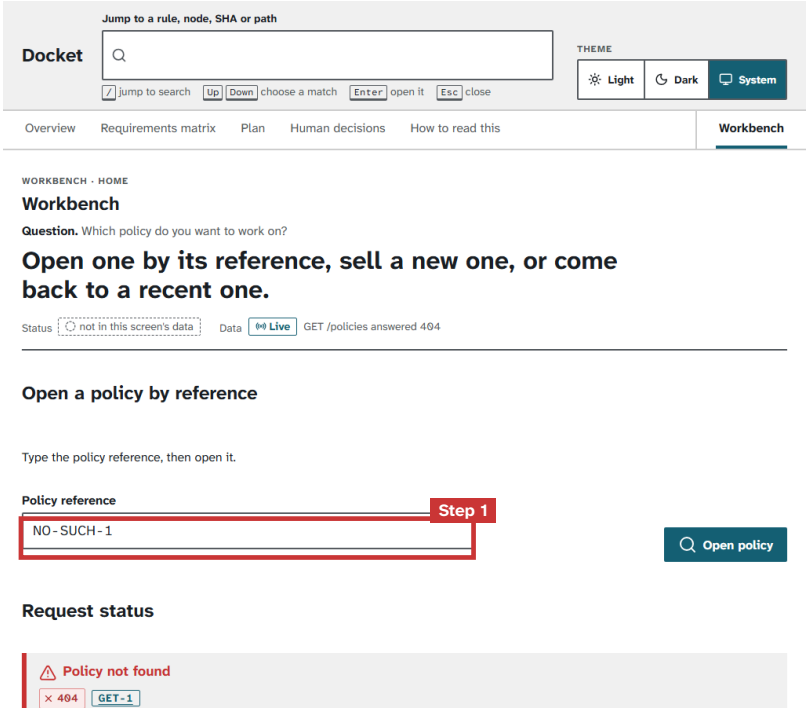
Step 2: **Sell this policy** with a start date in the past.



The API refuses a backdated start (SELL-2, 422): status, rule id, each violation. `#/workbench/sell` Requirement tags in view: **WB-3** **WB-1**

You should see: *Nothing was stored. The API answered 422, Start date outside the sale window.* The banner reads *The start date must be between 2026-10-25 and 2026-12-24* with the chip `SELL-2`. A 422 lists every violated rule.

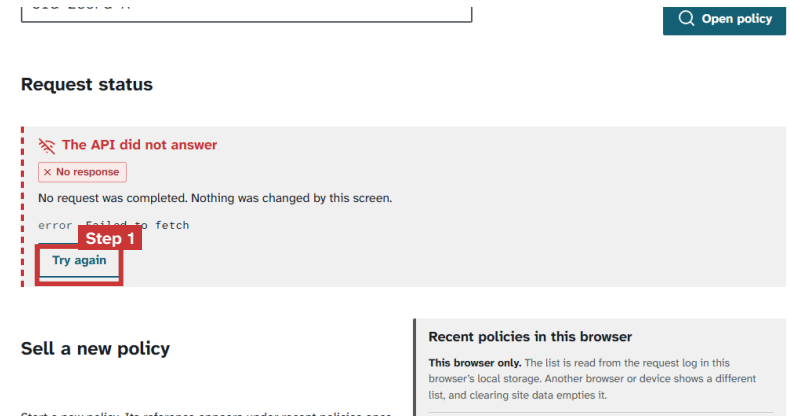
A 4xx: the policy does not exist



Workbench home, opening a policy that does not exist (GET-1, 404). `#/workbench` Requirement tags in view: **WB-2** **BUSY-1**

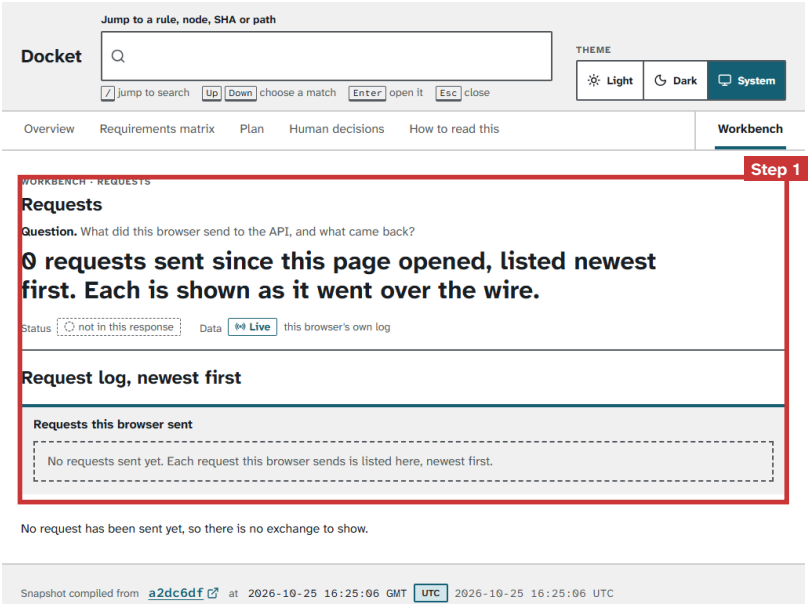
Step 1: Policy reference: `NO-SUCH-1`. The API answers **404** (`GET-1`) and the page says so under *Request status*.

No answer, and empty states



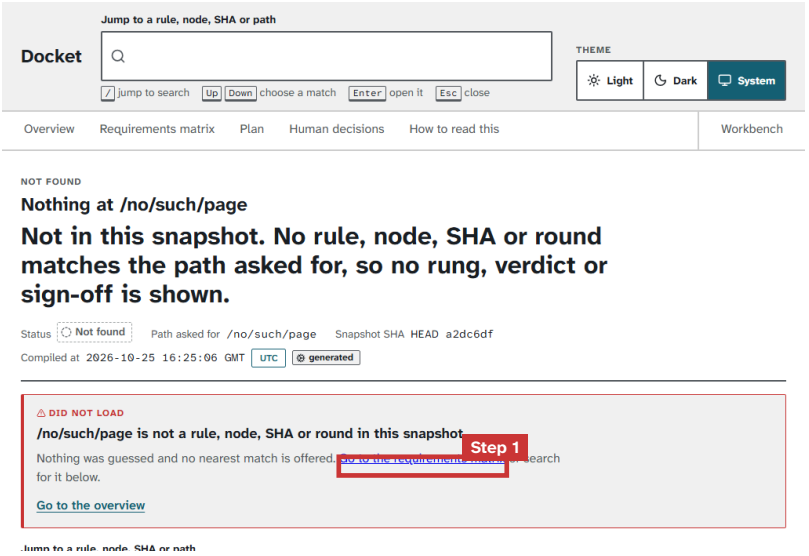
Workbench home, the API unreachable: the page says no request was completed. `#/workbench` Requirement tags in view: **BUSY-1** **WB-1** **WB-9**

Step 1: Try again: the API did not answer. *No request was completed. Nothing was changed by this screen.* The raw error, `Failed to fetch`, is shown as it came.



Request log before any request: an empty state. `#/workbench/requests` Requirement tags in view: **WB-7**

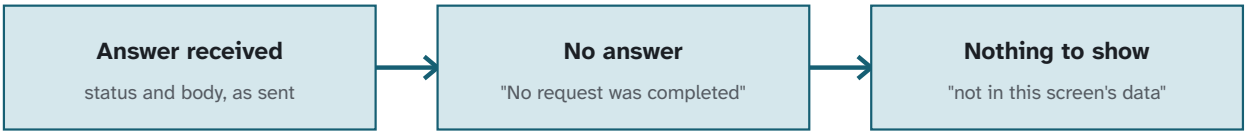
Step 1: The **request log**, empty: nothing has been sent, so it says so and invents no rows.



Not found: nothing is guessed, and there is a way home. `#/no/such/page` Requirement tags in view: **NAV-2**

Step 1: Go to the requirements matrix: the not-found page names the path asked for, offers no guess, and gives a way home.

> **Why**
> Choice: show the status the page received, and say plainly when none came. Alternative: translate every failure into a friendly message. Reason: a servicer who sees the API's own words and rule chip can tell a fault in the input from a fault in the service.



Honest states: what Docket shows when it has no answer

`src/Uinsure.Policies.Web/app/workbench/ProblemView.tsx:19`

```
export function ProblemView({ problem, controlId, focusKey, headingLevel = 3 }: ProblemViewProps) {
  if (problem.items.length > 0) {
    return (
      <ErrorSummary
        focusKey={focusKey}
        ruleHref={ruleHref}
```

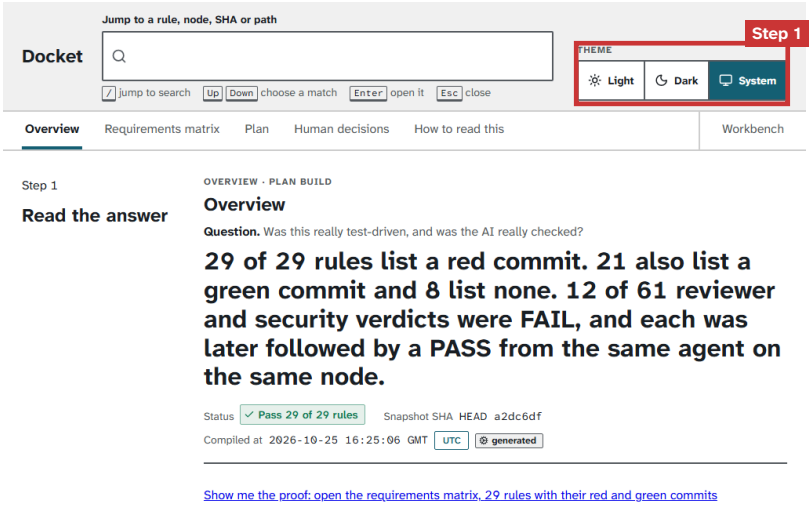
`src/Uinsure.Policies.Web/app/shell/NotFound.tsx:14`

```
export function NotFound({ asked }: { asked: string }) {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  return (
    <Region region="what-was-asked-for-and-where-to-go" req="NAV-2" label="Not found">
      <PageHeader
```

9 SJ9 · Make it yours

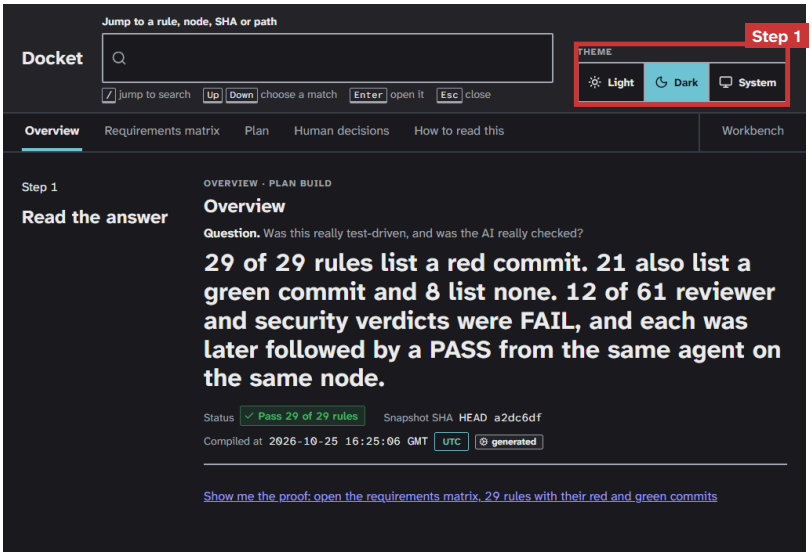
> **You decide** The theme, and which link to share.

> **Docket gives you** Light, dark and system themes from the header, and hash links that open any screen directly.



The Overview with the System choice, drawn in the light scheme. `#/` Requirement tags in view: **NAV-1**

Step 1: Theme control in the header: Light, Dark or System. *Step 1 Read the answer* printed on the page is the screen's own rung label, not this one.



The same screen after choosing Dark. `#/` Requirement tags in view: **NAV-1**

Step 1: Theme: *Dark* chosen. The choice sets `data-theme` on the page; System follows the operating system.

Every screen has a hash URL, so a screen can be bookmarked or sent to a colleague. Copy the address bar, or use these shapes:

To open	Link
A rule's evidence	`#/rules/CANCEL-1b`
A commit	`#/commits/<sha>`
A policy in the workbench	`#/workbench/policies/<reference>`
The request log	`#/workbench/requests`

`src/Uinsure.Policies.Web/app/shell/Frame.tsx:27`

```
function useAppliedTheme(choice: ThemeChoice): void {
  useEffect(() => {
    const system = window.matchMedia(DARK);
    const apply = (): void => {
      document.documentElement.dataset['theme'] = choice === 'system' ? (system.matches ? 'dark' :
'light') : choice;
    };
```

src/Uinsure.Policies.Web/app/shell/App.tsx:14

```
export function App() {
  return (
    <HashRouter>
      <Routes>
        <Route element={<Frame />}>
          <Route index element={<Overview />} />
```

Check your understanding

Question	Answer
How do I see what a cancellation will refund before I commit?	Quote the date on the cancel screen. It is read-only and shows each term's sum and rule.
What does Send anyway do?	It turns off the form's own checks so the API's rule answer can be seen.
Why does a new policy refuse renewal?	The window opens 30 days before the end date (RENEW-1, RENEW-2), a year away.
What does Docket give me that curl or Postman does not?	Forms that validate as you go, the refund before you commit, every call logged with a curl, and the rule behind each refusal.

Journey 2: the assessor, in the explorer

From one question, *was this really test-driven?*, to a rule's commits, a node's loop, a verdict's findings and the tests behind the screens.

You start at the overview's one-sentence answer, test it against the matrix, follow one rule from requirement to commit, walk a node, read a verdict, and look at the wider record. The last two steps go behind the screens: how the data is compiled and tested, and how the app serves a keyboard and both themes. Each rule is joined to its requirement, tests, commits and verdicts, so one click replaces a search.

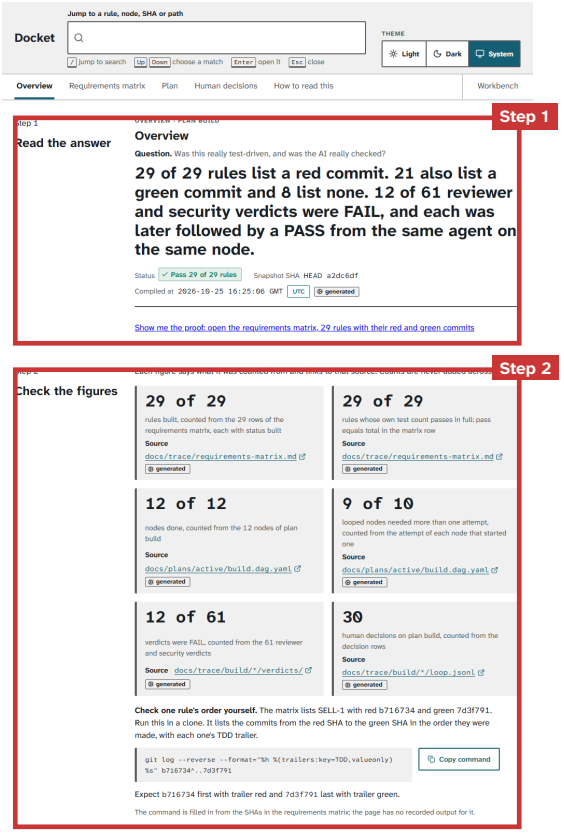
1 AJ1 · What Docket is and where to start

> **You decide** Which half is for you, and whether to run the app or read about it.

> **Docket gives you** The overview with the counts, the about page that explains how to read the trail, who each half is for, and the deployment evidence.

Half	For whom	Reads	Needs
Explorer, 17 screens	Assessors and developers checking how the API was built	`trace.json`	Nothing running
Workbench, 7 screens	Servicers and testers trying the API	The live API	The API on its port

To see it in five minutes, follow SJ1: build the app, preview it, and open `#/`.

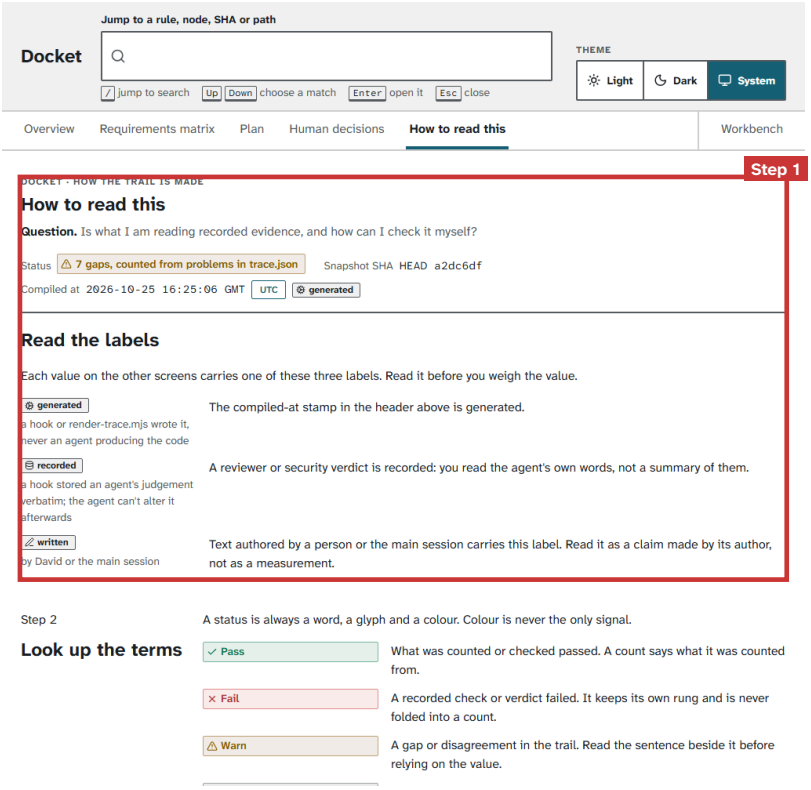


Overview: the answer and the counts. `#/` Requirement tags in view: `NAV-1` `NAV-1 VERIFY-1`

Step 1: Headline answer: one sentence, counted from the trail, that answers *Was this really test-driven, and was the AI really checked?* At this capture it reads 29 of 29 rules list a red commit, 21 also list a green commit and 8 list none.

Step 2: Receipt figures: each figure names what it was counted from and links to that source. Counts are never added across rules.

You should see: The screen's own rung labels read *Step 1 Read the answer* and *Step 2 Check the figures*. Here they happen to match the red labels in order; the red labels are this guide's.



How to read this: the labels, the glossary, re-verify with git. `#/about` Requirement tags in view: **DATA-2** **A11Y-1**

Step 1: Provenance labels: *generated* (a hook wrote it), *recorded* (stored verbatim) and *written* (a person's claim). Read the label before you weigh the value.

You should see: The page's own *Step 2 Look up the terms* explains each status word.

`src/Uinsure.Policies.Web/app/explorer/overview/Overview.tsx:134`

```
export function Overview() {
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
  const { trace, snapshot } = loaded;
  const plan = buildPlan(trace);
  return (
```

`src/Uinsure.Policies.Web/app/explorer/about/About.tsx:156`

```
export function About() {
  const { trace } = useShell();
  const gaps = trace.state === 'ready' ? gapsOf(trace.trace).length : null;
  return (
    <>
      {trace.state === 'ready' ? null : <DataNotice trace={trace} />}
```

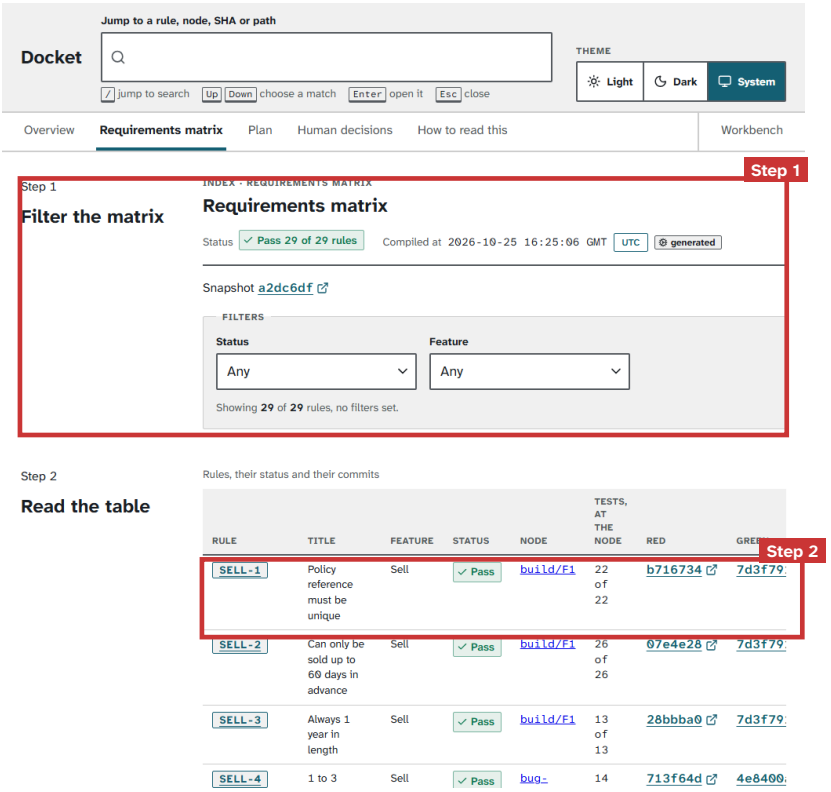
> **Why**
> Choice: Build a UI over the trail. Alternative: Read the git log and the markdown files. Reason: Each rule is already joined to its requirement, tests, commits and verdicts, so one click replaces a search.

> **Is the web part deployed?**
> No. The Static Web App is evidence only and was never deployed. The hosting configuration is infrastructure/web/staticwebapp.config.json:1; the Bicep module is `infrastructure/modules/staticwebapp.bicep`, and the pipeline's Web stage builds and tests the app. It was validated locally and `az` was never run.

2 AJ2 · Is every rule evidenced?

> **You decide** Which rule to look at, or whether to scan them all.

> **Docket gives you** The requirements matrix: every rule against its requirement, its status, its tests at the node that built it, and its red and green commits.

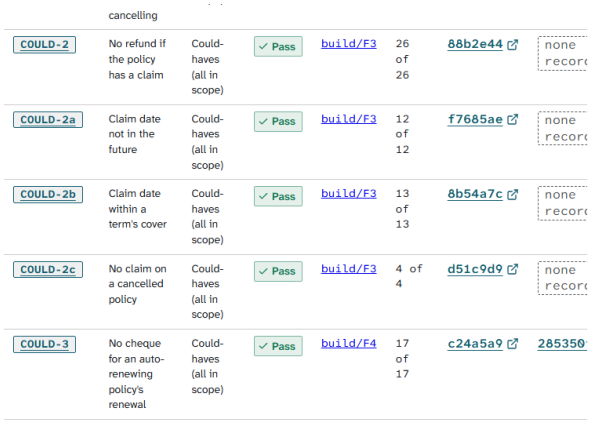


Requirements matrix: filters, then the table of rules against requirement, tests and commits. `#/rules` Requirement tags in view: **MATRIX-1**

Step 1: Filters: Status and Feature narrow the table. The count under them reads *Showing 29 of 29 rules*.

Step 2: The SELL-1 row: rule, title, feature, status, node, tests at the node (22 of 22), then the red and green commit links.

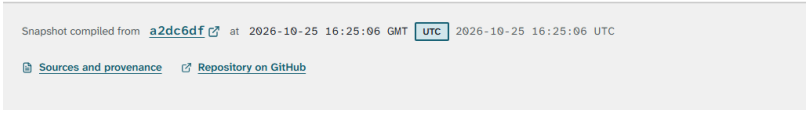
You should see: The screen's own rungs read *Step 1 Filter the matrix* and *Step 2 Read the table*; they match the red labels here.



Step 3

Test counts are per node

The matrix counts a rule's tests at the node that built it, while the rule page lists every test traited with it, so the two figures can differ. For example SELL-1 has 22 of 22 at F1.



The matrix down to its last row and the scope note. `#/rules` Requirement tags in view: **MATRIX-1** **DATA-4**

You should see: The last rows run to `COULD-3`. The page's own *Step 3 Test counts are per node* explains why a rule's count here can differ from its rule page: the matrix counts the tests at the node that built it, while the rule page lists every test traited with it.

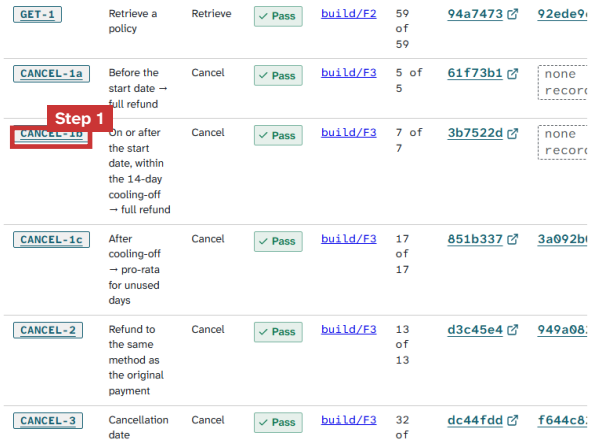
`src/Uinsure.Policies.Web/app/explorer/matrix/Matrix.tsx:166`

```
export function Matrix() {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  return <MatrixTables trace={trace.trace} snapshotSha={trace.snapshot.sha} snapshotShort=
{trace.snapshot.short} utc={trace.snapshot.utc} />;
}
```

3 AJ3 · Follow one rule

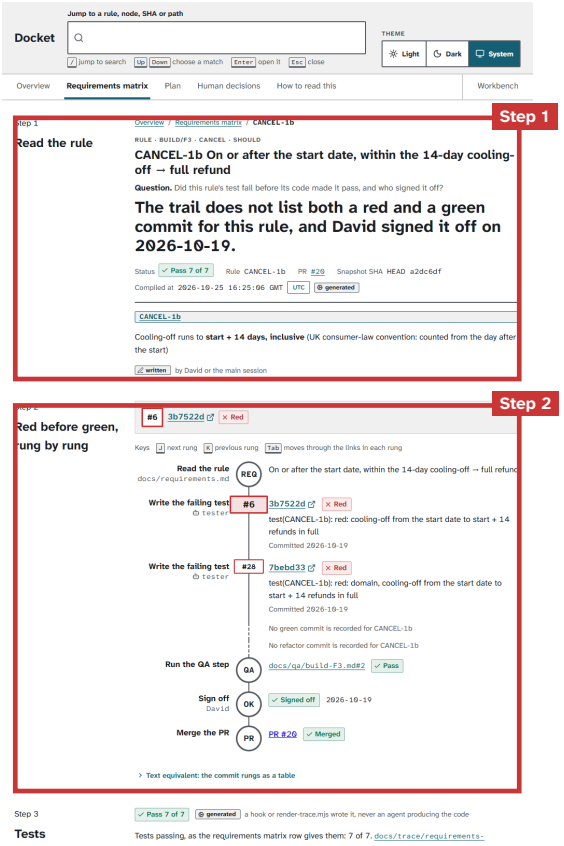
You decide CANCEL-1b, or any other rule.

Docket gives you The rule page: the requirement and its agreed interpretation, the red-before-green evidence rung by rung, the verification recipe, then a commit with its trailers.



Requirements matrix, the CANCEL-1b row. `#/rules` Requirement tags in view: **MATRIX-1**

Step 1: The **CANCEL-1b** link in the matrix opens the rule page. Typing `CANCEL-1b` in the header's jump-to-search box and pressing Enter does the same.

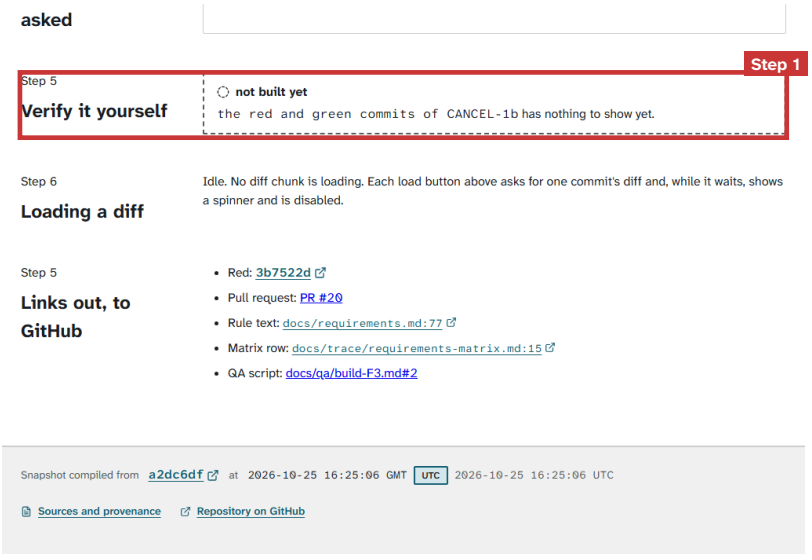


One rule: the requirement and its interpretation, then red before green. `#/rules/CANCEL-1b` Requirement tags in view: **RULE-1** **RULE-2**

Step 1: Rule text and agreed interpretation: cooling-off runs to start + 14 days, inclusive. The label *written* says a person wrote it.

Step 2: Red before green: the rungs read down from the requirement to the red test commits (3b7522d, 7bebd33), the QA step, David's sign-off and the merged pull request.

You should see: The screen's own rungs are *Step 1 Read the rule* and *Step 2 Red before green, rung by rung*; they match the red labels. The page is honest about a gap: it says *No green commit is recorded for CANCEL-1b* and that the trail does not list both a red and a green commit for it.

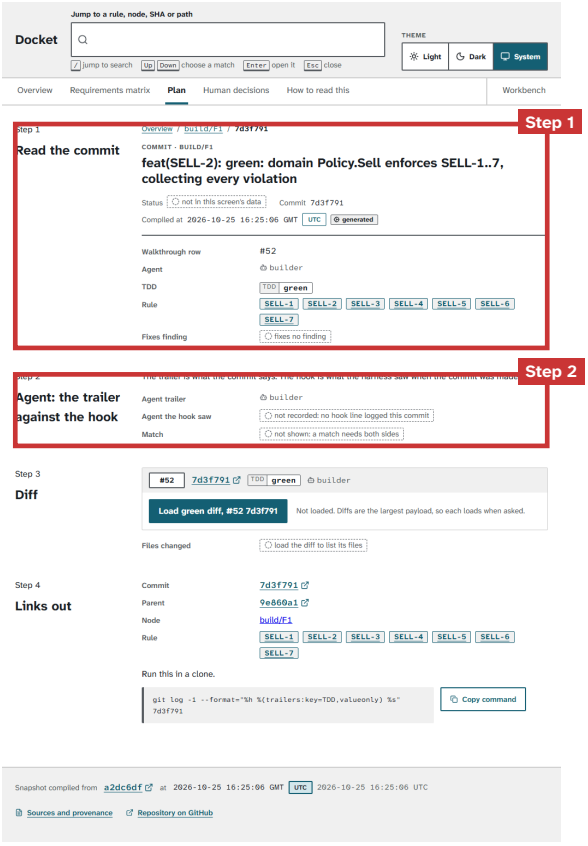


The rule page's end: verify it yourself, and the links out. `#/rules/CANCEL-1b` Requirement tags in view: **VERIFY-1** **BUSY-1** **NAV-3**

Step 1: Verify it yourself: for a rule whose trail has both a red and a green commit this rung gives the `git log` recipe that the Overview shows for SELL-1. For CANCEL-1b it says *not built yet*, because there is no green commit to name.

You should see: The page's own rung labels at this end read *Step 5 Verify it yourself*, *Step 6 Loading a diff* and *Step 5 Links out, to GitHub*. They belong to the screen. Links out: the red commit, the pull request, the rule text and the matrix row.

Follow a commit link, for example one of the builder's commits, to its page.

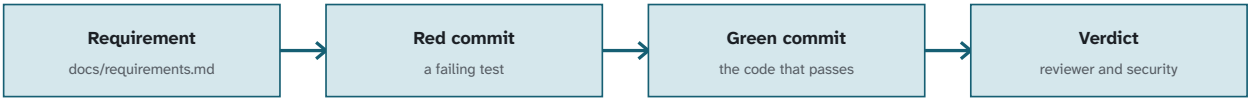


One commit: header and trailers, the agent, the diff, the links out. `#/commits/7d3f7914e32e01f4a4a3d4a6d75dcfe60f94d668` Requirement tags in view: **NAV-2** **AGENT-1** **RULE-3** **NAV-3**

Step 1: Commit header and trailers: the walkthrough row, the agent (*builder*), the TDD step (*green*) and each rule it names.

Step 2: Trailer agent against hook agent: what the commit says against what the harness saw. Here it reads *not recorded: no hook line logged this commit*, so no match is claimed.

You should see: The screen's own rungs are *Step 1 Read the commit* and *Step 2 Agent: the trailer against the hook*, then *Step 3 Diff* (loaded on demand) and *Step 4 Links out*.



Evidence chain: how a rule is proved

`src/Uinsure.Policies.Web/app/explorer/rule/Rule.tsx:309`

```
export function Rule() {
  const { ruleId = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const rule = trace.trace.rules.find((candidate) => candidate.id === ruleId);
  if (rule === undefined) return <NotFound asked={`/rules/${ruleId}`} />;
```

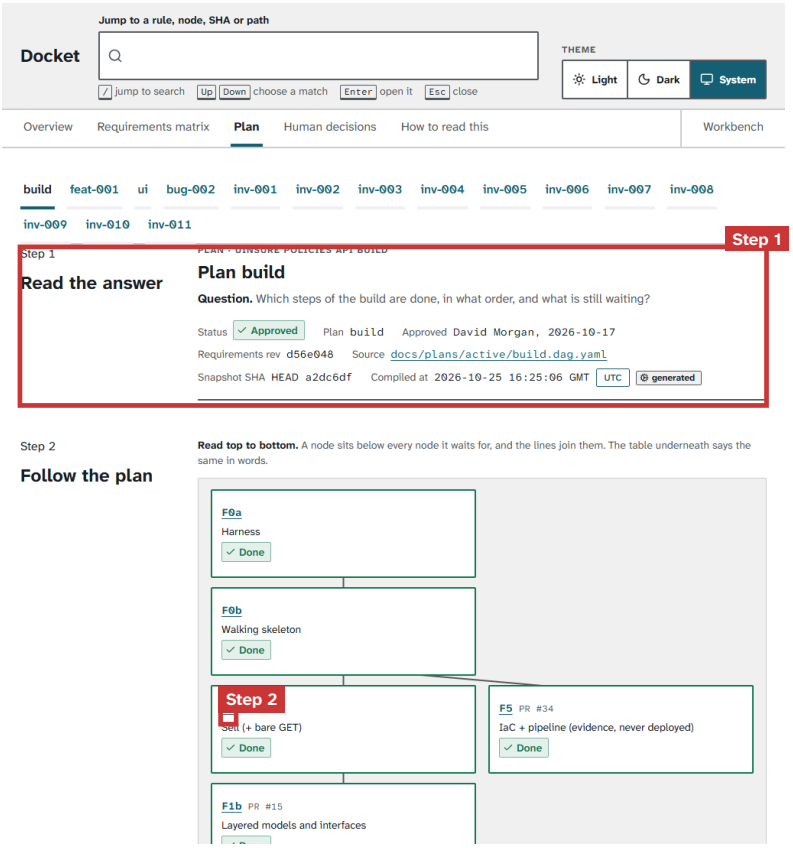
`src/Uinsure.Policies.Web/app/explorer/commit/Commit.tsx:83`

```
export function Commit() {
  const { sha = '' } = useParams();
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
  const commit = sha.length < 7 ? undefined : loaded.trace.commits.find((candidate) =>
candidate.sha.startsWith(sha));
  if (commit === undefined) return <NotFound asked={`/commits/${sha}`} />;
```

4 AJ4 · Was it built test-first?

> **You decide** Which node to walk.

> **Docket gives you** The plan as a picture, then the node, its walkthrough, its loop and its activity: red, stub and green commits in order.

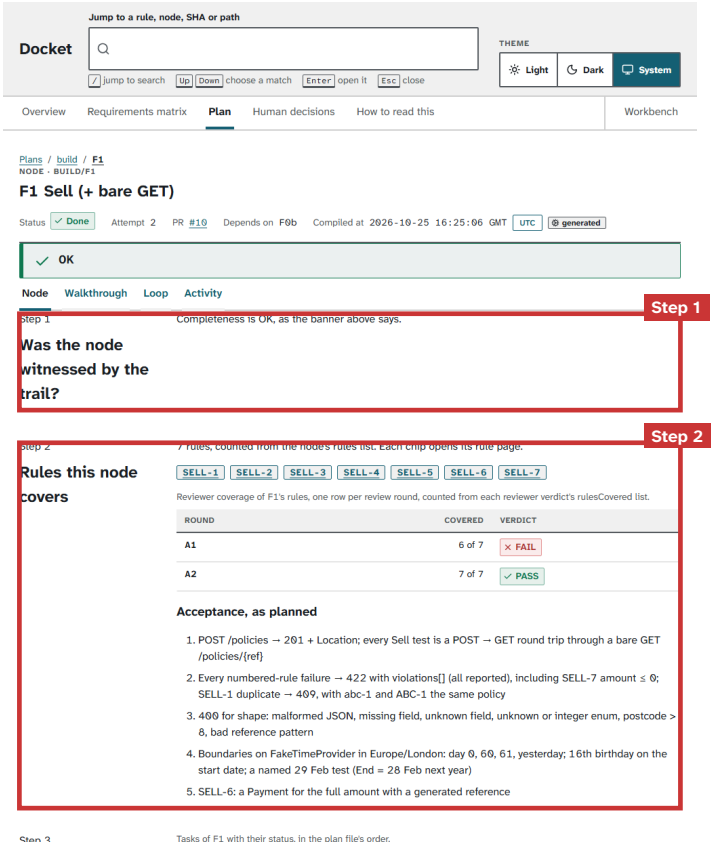


A plan: the picture of its nodes. `#/plans/build` Requirement tags in view: **DAG-1**

Step 1: Plan header: *Approved*, by whom and when, the requirements revision and the source file.

Step 2: The plan picture: nodes sit below the nodes they wait for. Choose the `F1` node.

You should see: The screen's own rungs are *Step 1 Read the answer* and *Step 2 Follow the plan*. The table under the picture says the same in words.

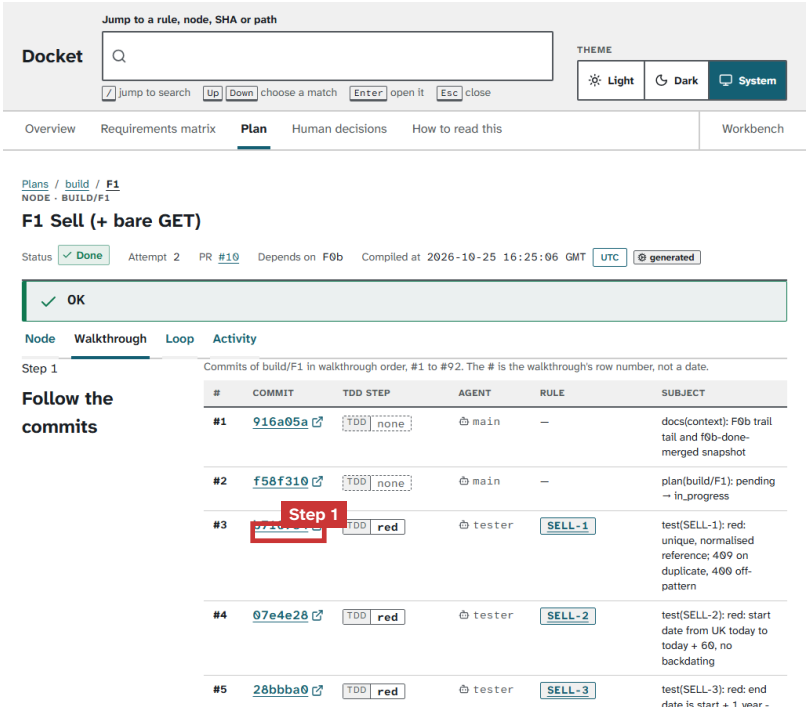


A node: the rules it covers, its tasks and its attempts. `#/plans/build/F1` Requirement tags in view: **DATA-4** **DATA-8** **NODE-1**

Step 1: Completeness banner: *OK*, so the trail witnessed the node.

Step 2: Rules this node covers: SELL-1 to SELL-7, and the reviewer's coverage by round (A1 covered 6 of 7 and failed; A2 covered 7 of 7 and passed).

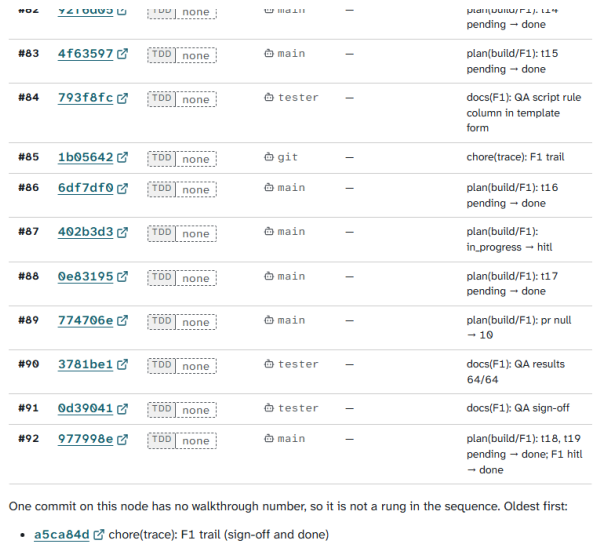
You should see: The screen's own rungs are *Step 1 Was the node witnessed by the trail?* and *Step 2 Rules this node covers*.



A node's walkthrough: the commits in order. `#/plans/build/F1/commits` Requirement tags in view: **TDD-1**

Step 1: The first red commit: row #3, `b716734`, the tester's failing test for SELL-1. The # is the walkthrough row, not a date. Rows #4 and #5 are the red tests for SELL-2 and SELL-3.

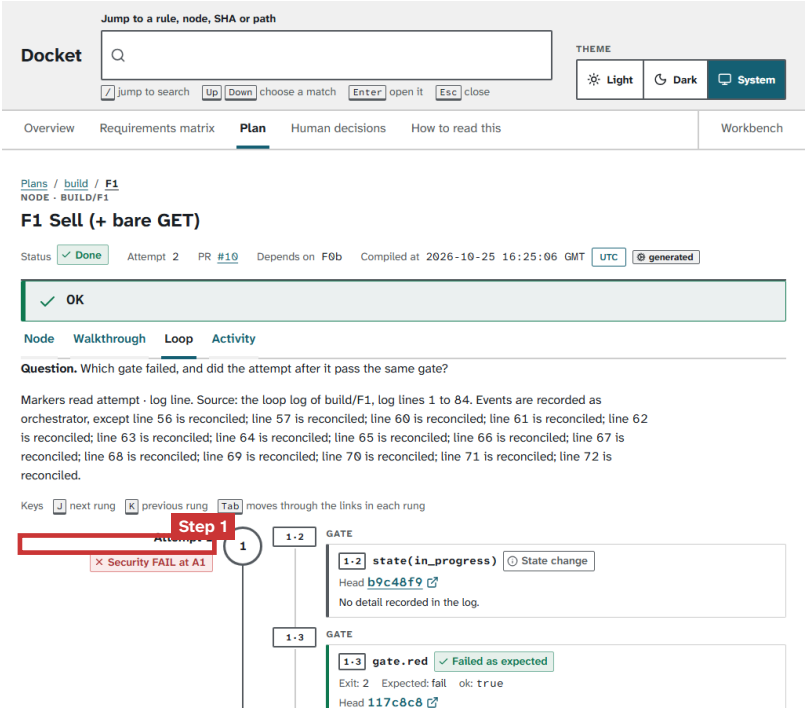
You should see: The screen's own *Step 1 Follow the commits* is this table.





The walkthrough to its last rows. `#/plans/build/F1/commits` Requirement tags in view: **TDD-1**

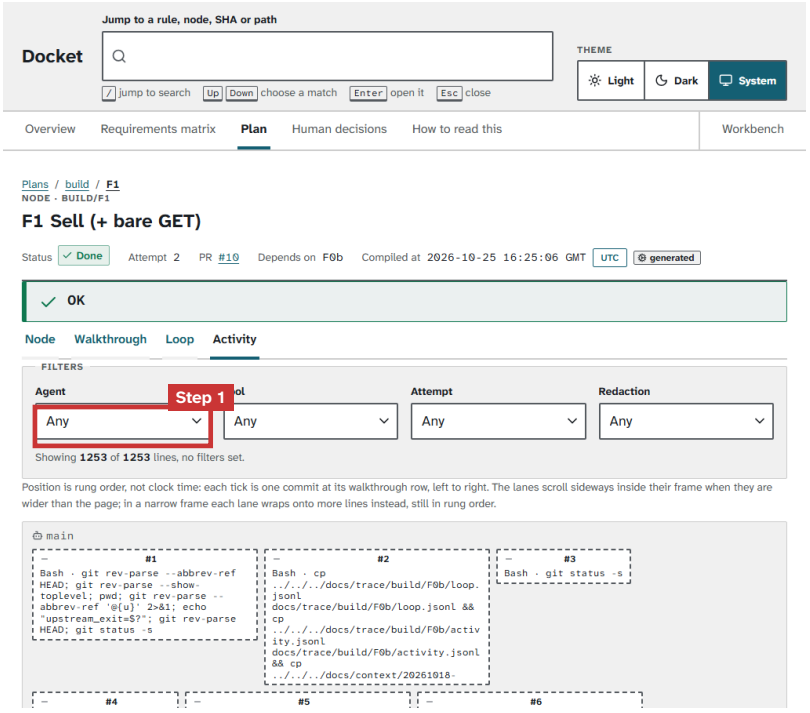
You should see: The sequence ends at #92. The closing rows are plan updates by *main*, QA results by the tester and a trail commit by *git*, tagged *TDD none*. One commit has no walkthrough number and is listed apart, oldest first. Red, stub and green commits sit earlier in the same list, in order; the builder's green commit for SELL-2 is #52.



A node's loop: attempt by attempt. `#/plans/build/F1/loop` Requirement tags in view: **LOOP-2**

Step 1: Attempt 1, the first rung of the ladder: it ended in a security FAIL at A1. The markers read attempt then loop-log line, so *1·2* is attempt 1, line 2.

You should see: The question on this tab is *Which gate failed, and did the attempt after it pass the same gate?*



A node's activity: one lane per agent. `#/plans/build/F1/activity` Requirement tags in view: **AGENT-2**

Step 1: Agent filter: choose an agent to narrow the lanes. The Tool, Attempt and Redaction filters sit beside it, and the page says *Showing 1253 of 1253 lines*.

You should see: Below the filters is one lane per agent, the first being *main*, with one tick per commit in rung order. A text table beneath is the equivalent for the keyboard and screen readers.

`src/Uinsure.Policies.Web/app/explorer/plan/Plan.tsx:32`

```
export function Plan() {
  const { planId = '' } = useParams();
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
  const { trace, snapshot } = loaded;
  const plan = trace.plans.find((candidate) => candidate.id === planId) as InvestigationPlan |
undefined;
```

`src/Uinsure.Policies.Web/app/explorer/node/Node.tsx:27`

```
export function Node() {
  const { planId = '', nodeId = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const node = trace.trace.nodes.find((candidate) => candidate.plan === planId && candidate.id ===
nodeId);
  if (node === undefined) return <NotFound asked={`/plans/${planId}/${nodeId}`} />;
```

`src/Uinsure.Policies.Web/app/explorer/walkthrough/Walkthrough.tsx:43`

```
export function Walkthrough() {
  const { planId = '', nodeId = '' } = useParams();
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
  const { trace } = loaded;
  const node = trace.nodes.find((candidate) => candidate.plan === planId && candidate.id === nodeId);
```

src/Uinsure.Policies.Web/app/explorer/loop/Loop.tsx:63

```
export function Loop() {
  const { planId = '', nodeId = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const node = trace.trace.nodes.find((candidate) => candidate.plan === planId && candidate.id ===
nodeId);
  if (node === undefined) return <NotBuiltYet kind="node" name={`${planId}/${nodeId}`} back={{ href:
'#/', label: 'Back to the overview' }} />;
```

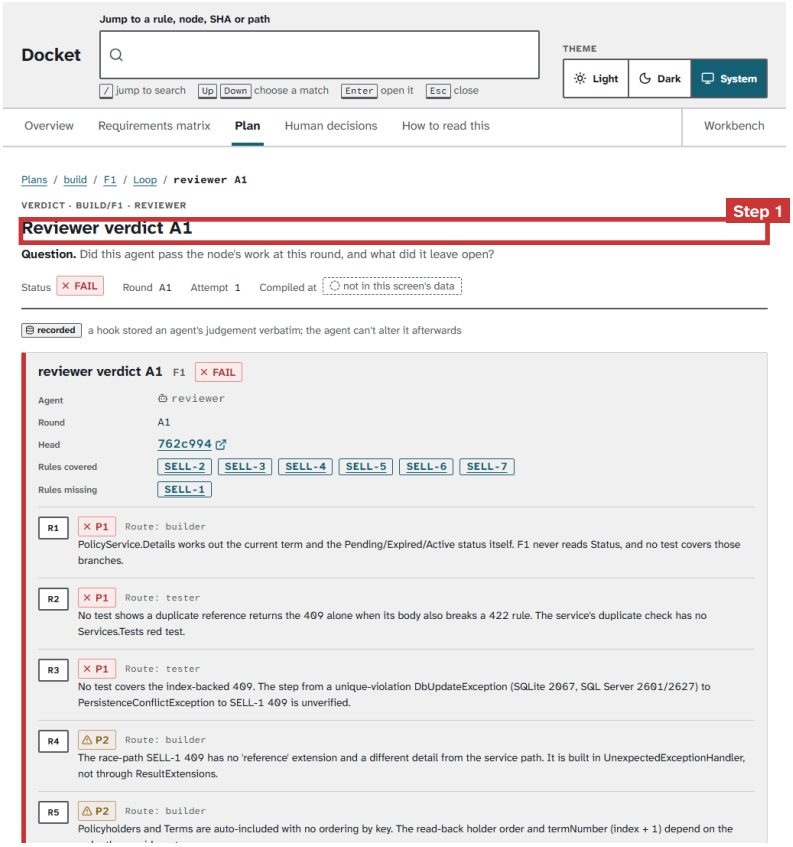
src/Uinsure.Policies.Web/app/explorer/activity/Activity.tsx:119

```
export function Activity() {
  const { trace } = useShell();
  const { planId, nodeId } = useParams();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  if (planId === undefined || nodeId === undefined) return null;
  return <ActivityBody trace={trace.trace} plan={planId} node={nodeId} />;
```

5 AJ5 · Read a verdict and its findings

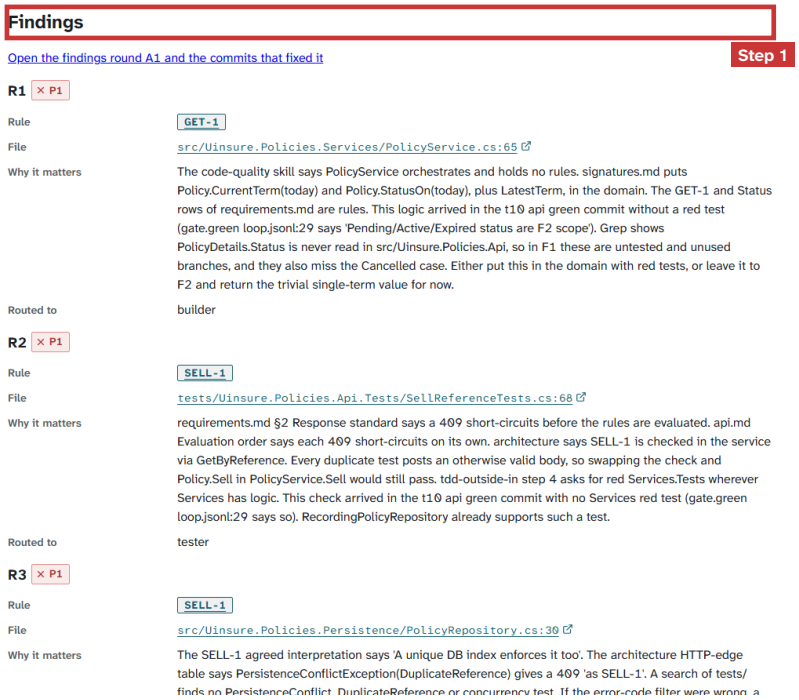
You decide Whether a node passed, and which finding to follow.

Docket gives you The verdict, each finding with its priority and route, then the findings round and the commit that fixed each.



A verdict: the summary, then each finding. `#/plans/build/F1/verdicts/A1/reviewer` Requirement tags in view: **LOOP-3**

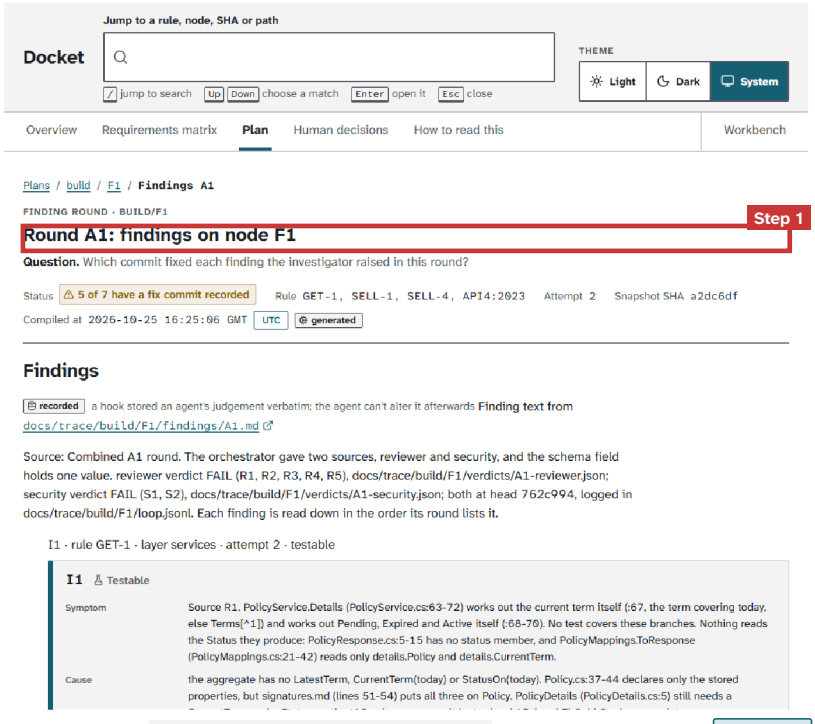
Step 1: The verdict's title and status: *Reviewer verdict A1*, with *FAIL*, round A1 and attempt 1 beneath it. The label *recorded* further down says a hook stored the agent's words verbatim.



The verdict's findings, further down. `#/plans/build/F1/verdicts/A1/reviewer` Requirement tags in view: **LOOP-3**

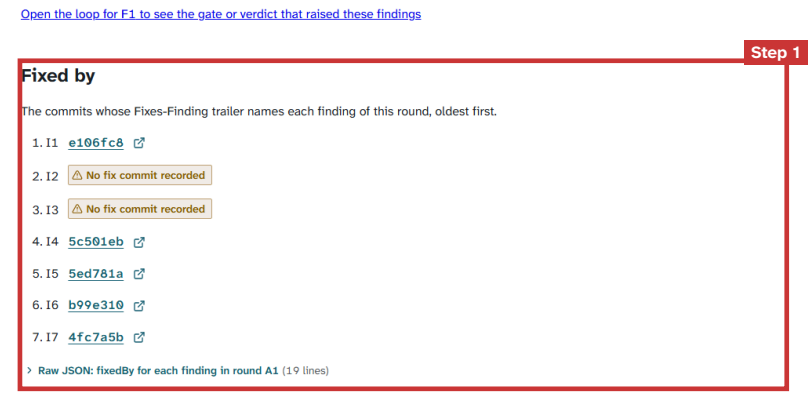
Step 1: The **Findings** heading. Its first link, *Open the findings round A1 and the commits that fixed it*, leads to the investigator's round.

You should see: Findings R1, R2 and R3 are P1 (they fail the node), each with its rule, file and route (builder or tester).



A findings round. `#/plans/build/F1/findings/A1` Requirement tags in view: **LOOP-4**

Step 1: **The round's title**: *Round A1: findings on node F1*. Its status warns that *5 of 7 have a fix commit recorded*; the page does not hide the two without one.



The findings round, the end: the commits that fixed it. `#/plans/build/F1/findings/A1` Requirement tags in view: **LOOP-4** **LOOP-2**

Step 1: Fixed by: the commits whose `Fixes-Finding` trailer names each finding, oldest first. I2 and I3 say *No fix commit recorded*.

`src/Uinsure.Policies.Web/app/explorer/verdict/Verdict.tsx:182`

```
export function Verdict() {
  const { planId = '', nodeId = '', round = '', agent = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const verdict = verdictsOf(trace.trace, planId, nodeId).find((v) => v.round === round && v.agent ===
agent);
  if (verdict === undefined) {
```

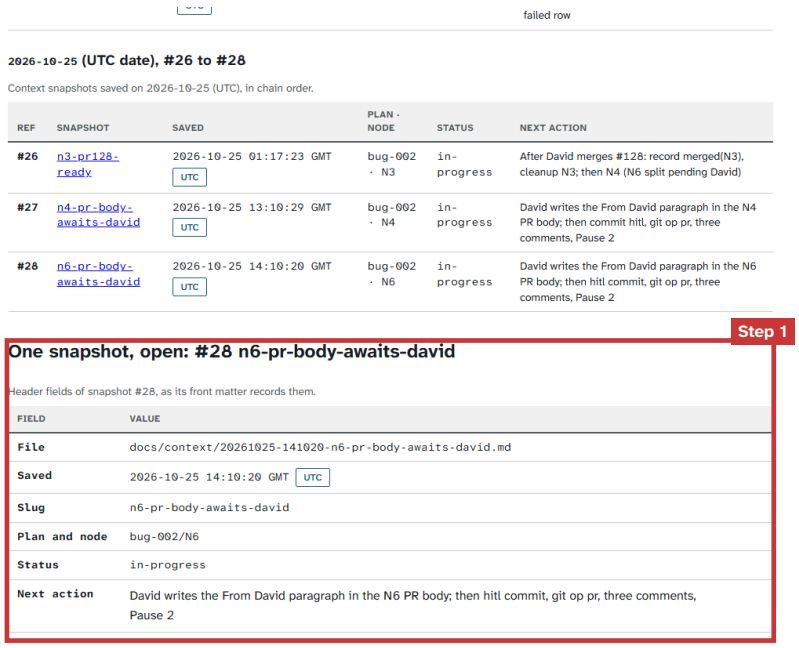
`src/Uinsure.Policies.Web/app/explorer/finding/Finding.tsx:124`

```
export function Finding() {
  const { planId = '', nodeId = '', round = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const findings = findingsOf(trace.trace, planId, nodeId, round);
  if (findings.length === 0) {
```

6 AJ6 · The wider story

> **You decide** What to read next: the sessions, the harness, the decisions or the numbers.

> **Docket gives you** The sessions timeline, the agents and hooks of the harness, the human decisions with their evidence line, and the stats per agent and node.



failed row

2026-10-25 (UTC date), #26 to #28

Context snapshots saved on 2026-10-25 (UTC), in chain order.

REF	SNAPSHOT	SAVED	PLAN · NODE	STATUS	NEXT ACTION
#26	n3-pr128-ready	2026-10-25 01:17:23 GMT UTC	bug-002 · N3	in-progress	After David merges #128: record merged(N3), cleanup N3; then N4 (N6 split pending David)
#27	n4-pr-body-awaits-david	2026-10-25 13:10:29 GMT UTC	bug-002 · N4	in-progress	David writes the From David paragraph in the N4 PR body; then commit hitl, git op pr, three comments, Pause 2
#28	n6-pr-body-awaits-david	2026-10-25 14:10:20 GMT UTC	bug-002 · N6	in-progress	David writes the From David paragraph in the N6 PR body; then hitl commit, git op pr, three comments, Pause 2

Step 1

One snapshot, open: #28 n6-pr-body-awaits-david

Header fields of snapshot #28, as its front matter records them.

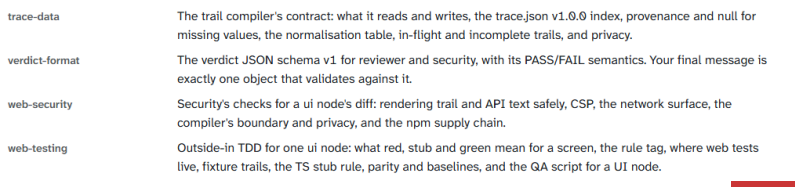
FIELD	VALUE
File	docs/context/20261025-141020-n6-pr-body-awaits-david.md
Saved	2026-10-25 14:10:20 GMT UTC
Slug	n6-pr-body-awaits-david
Plan and node	bug-002/N6
Status	in-progress
Next action	David writes the From David paragraph in the N6 PR body; then hitl commit, git op pr, three comments, Pause 2

Snapshot compiled from a2dc6df at 2026-10-25 16:25:06 GMT UTC 2026-10-25 16:25:06 UTC

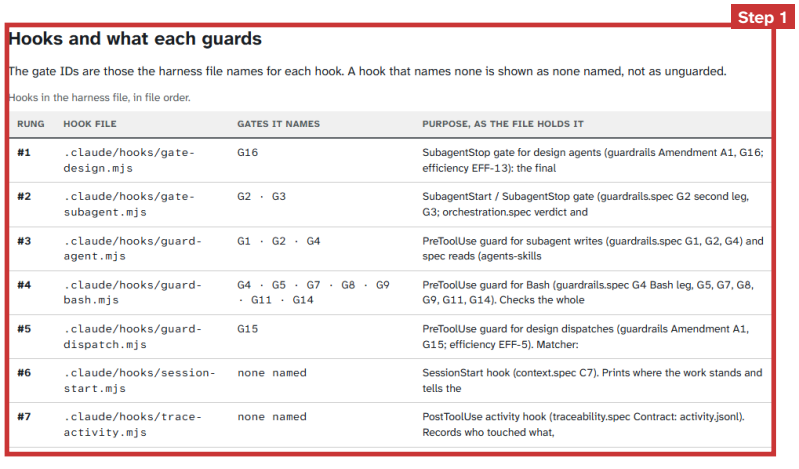
Sources and provenance Repository on GitHub

Sessions, down to the latest day with one snapshot open. `#/sessions` Requirement tags in view: **CTX-1** **NAV-2**

Step 1: One snapshot open: its file, when it was saved, its plan and node, and the next action the hand-off named. It has its own URL, `#/sessions/<file>`.



trace-data	The trail compiler's contract: what it reads and writes, the trace.json v1.0.0 index, provenance and null for missing values, the normalisation table, in-flight and incomplete trails, and privacy.
verdict-format	The verdict JSON schema v1 for reviewer and security, with its PASS/FAIL semantics. Your final message is exactly one object that validates against it.
web-security	Security's checks for a ui node's diff: rendering trail and API text safely, CSP, the network surface, the compiler's boundary and privacy, and the npm supply chain.
web-testing	Outside-in TDD for one ui node: what red, stub and green mean for a screen, the rule tag, where web tests live, fixture trails, the TS stub rule, parity and baselines, and the QA script for a UI node.



Step 1

Hooks and what each guards

The gate IDs are those the harness file names for each hook. A hook that names none is shown as none named, not as unguarded.

Hooks in the harness file, in file order.

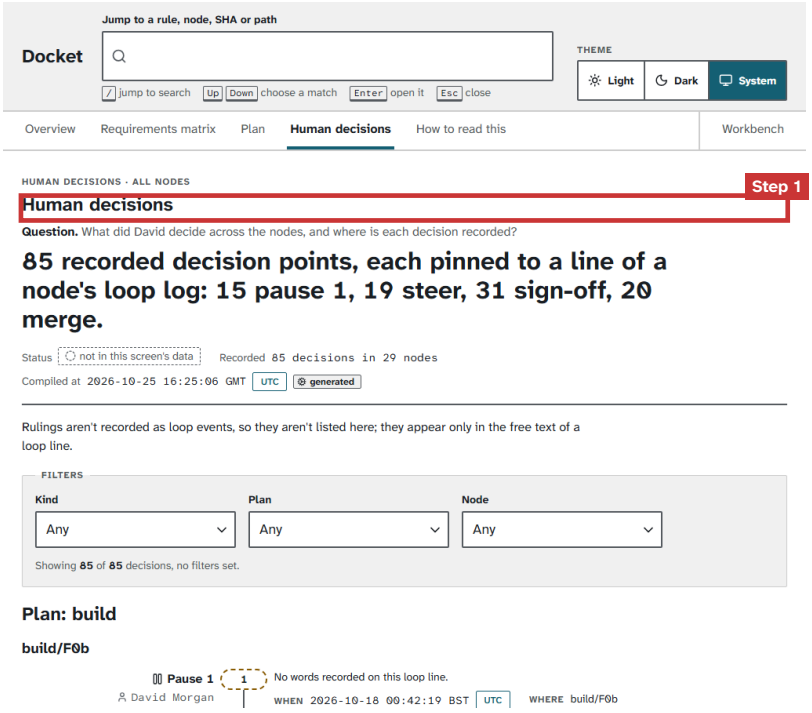
RUNG	HOOK FILE	GATES IT NAMES	PURPOSE, AS THE FILE HOLDS IT
#1	.claude/hooks/gate-design.mjs	G16	SubagentStop gate for design agents (guardrails Amendment A1, G16; efficiency EFF-13): the final
#2	.claude/hooks/gate-subagent.mjs	G2 · G3	SubagentStart / SubagentStop gate (guardrails.spec G2 second leg, G3; orchestration.spec verdict and
#3	.claude/hooks/guard-agent.mjs	G1 · G2 · G4	PreToolUse guard for subagent writes (guardrails.spec G1, G2, G4) and spec reads (agents-skills
#4	.claude/hooks/guard-bash.mjs	G4 · G5 · G7 · G8 · G9 · G11 · G14	PreToolUse guard for Bash (guardrails.spec G4 Bash leg, G5, G7, G8, G9, G11, G14). Checks the whole
#5	.claude/hooks/guard-dispatch.mjs	G15	PreToolUse guard for design dispatches (guardrails Amendment A1, G15; efficiency EFF-5). Matcher:
#6	.claude/hooks/session-start.mjs	none named	SessionStart hook (context.spec C7). Prints where the work stands and tells the
#7	.claude/hooks/trace-activity.mjs	none named	PostToolUse activity hook (traceability.spec Contract: activity.jsonl). Records who touched what,

Snapshot compiled from a2dc6df at 2026-10-25 16:25:06 GMT UTC 2026-10-25 16:25:06 UTC

Sources and provenance Repository on GitHub

The harness, down to its hooks. `#/harness` Requirement tags in view: **AGENT-3**

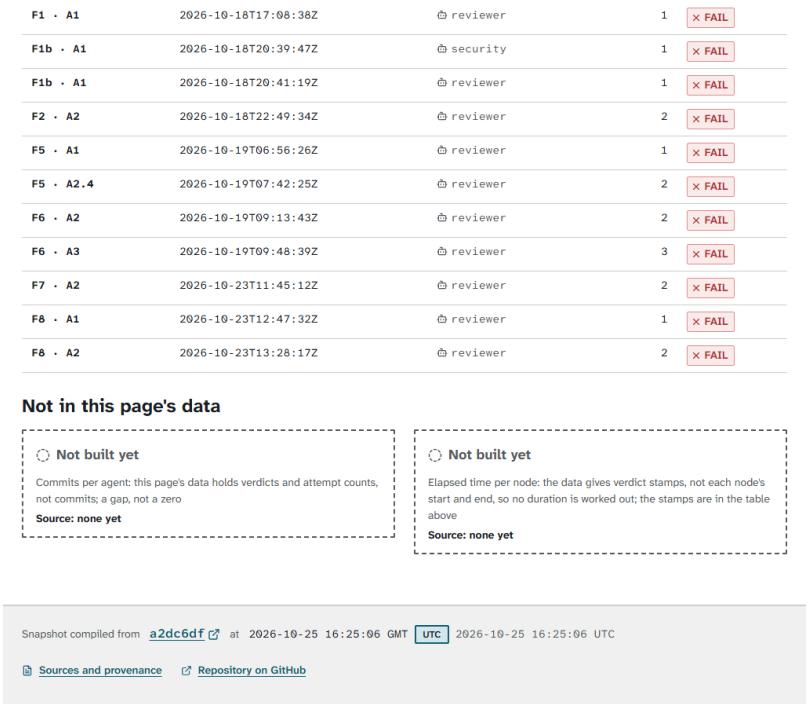
Step 1: Hooks and what each guards: the hook file, the gate ids it names (or *none named*, not *unguarded*) and its purpose, as the file holds it.



Human decisions, by plan. `#/decisions` Requirement tags in view: **HITL-1**

Step 1: The decisions page, *Human decisions*. Its headline counts 85 recorded decision points, each pinned to a line of a node's loop log. Filter by kind, plan or node.

You should see: The page says plainly that rulings are not recorded as loop events, so they are not listed here.



Stats, the end: what is not in the page's data. `#/stats` Requirement tags in view: **STATS-1**

You should see: A table of FAIL verdicts by node, round and agent, then two boxes marked *Not built yet*: commits per agent and elapsed time per node. They are gaps, drawn as gaps, not zeroes.

src/Uinsure.Policies.Web/app/explorer/sessions/Sessions.tsx:52

```
export function Sessions() {
  const { trace } = useShell();
  const { file } = useParams();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const rows: Row[] = trace.trace.sessions.map((session, index) => ({ ...session, id: session.file,
position: index + 1 }));
  const latest = rows[rows.length - 1];
```

src/Uinsure.Policies.Web/app/explorer/harness/Harness.tsx:30

```
export function Harness() {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const { harness } = trace.trace;
  const agents: Agent[] = (harness?.agents ?? []).map((agent, index) => ({ ...agent, id: agent.file,
position: index + 1 }));
  const hooks: Hook[] = (harness?.hooks ?? []).map((hook, index) => ({ ...hook, id: hook.file,
position: index + 1 }));
```

src/Uinsure.Policies.Web/app/explorer/decisions/Decisions.tsx:42

```
export function Decisions() {
  const { trace } = useShell();
  const [filters, setFilters] = useState<Filters>(NO_FILTERS);
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const all = trace.trace.decisions.filter((d) => kindOf(d) !== null);
  const shown = all.filter((d) => passes(d, filters));
```

src/Uinsure.Policies.Web/app/explorer/stats/Stats.tsx:161

```
export function Stats() {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  return <StatsTables trace={trace.trace} snapshot={trace.snapshot} />;
}
```

7 AJ7 · Behind the screens

> **You decide** How deep to go.

> **Docket gives you** The folder tree, the compiler and `trace.json`, the kit and its tokens, the design pipeline and the screen contracts, and the kinds of test, with a real run of each.

The folder tree after U10

The web project and its tests, as git tracks them. `dist`, `node_modules` and `test-results` appear after a build, an install or a test run and are git-ignored.

1. List the folders (run on this checkout).

```
git ls-tree --name-only HEAD <folder>/
```

You should see: One block per folder.

```
src/Uinsure.Policies.Web/
  .gitignore  .nvmrc  .stylelintrc.json  .vscode  Docket.Web.esproj  app  compiler
eslint.config.js  index.html  kit  kitchen-sink.html  package-lock.json  package.json
playwright.config.ts  scripts  tsconfig.json  vite.config.ts
src/Uinsure.Policies.Web/app/
  data  explorer  main.tsx  shell  workbench
src/Uinsure.Policies.Web/app/explorer/
  Ladder.tsx  about  activity  commit  decisions  diff  explorer.css  finding  harness  loop
loopData.ts  matrix  node  overview  plan  rule  ruleFacts.ts  sessions  stats  u1Routes.tsx
u2Routes.tsx  u3Routes.tsx  u5.css  u5Routes.tsx  verdict  walkthrough
src/Uinsure.Policies.Web/app/workbench/
  ProblemView.tsx  RequestsDrawer.tsx  browser.ts  cancel  claim  home  policy  renew  requests
routes.tsx  sell  statusTone.ts  workbench.css
src/Uinsure.Policies.Web/kit/
  index.ts  layouts  patterns  primitives  project  sink  tokens
src/Uinsure.Policies.Web/compiler/
  git.ts  index.ts  plans.ts  privacy.ts  test-sources.ts  trace-files.ts
tests/Uinsure.Policies.Web.Tests/
  e2e  fixtures  kit  unit
tests/Uinsure.Policies.Web.Tests/e2e/
  N2  N3  N4  U0b  U1  U10  U2  U3  U4  U5  U7  U8  W1  W2  W3  regression
docs/ui/
  baselines  components.md  concept.md  derived  directions.json  fixture  flows.md  kit  mockups
research  screens.json  screens  stylekit  styles.md
```

The compiler that builds `trace.json`

`src/Uinsure.Policies.Web/compiler` reads git and the trail files at one ref, scrubs private data, and writes `public/trace.json`. The explorer reads only that file. The dev server compiles it at start and answers 503 *compiling* until it is ready.

1. Compile the trail (the step you ran in SJ1).

```
npm --prefix src/Uinsure.Policies.Web run trace
```

You should see: Exit 0 in 76.4 s.

```
> trace
> node compiler/index.ts --ref HEAD
```

`src/Uinsure.Policies.Web/compiler/index.ts:16`

```
export interface CompileOptions {
  root: string;
  ref: string;
  now?: () => Date;
}
```

The kit and its tokens

`src/Uinsure.Policies.Web/kit` holds the primitives, patterns and layouts that every screen is built from, with `tokens/tokens.css` for colour, type and spacing. This guide uses the same tokens, copied unedited into `docs/guides/assets`. `kitchen-sink.html` shows every component.

The design pipeline, and why screens are contracts

`docs/ui` holds the pipeline: directions, concept, styles, mockups, then one static HTML per screen state in `docs/ui/screens`, with signed-off shots in `docs/ui/screens/_shots` and baselines in `docs/ui/baselines`.

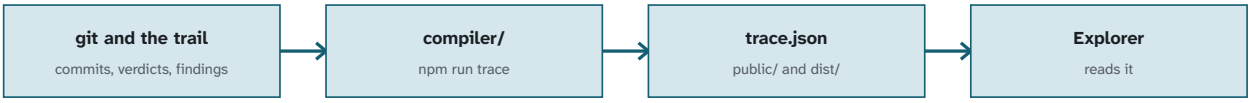
> **Why**
> Choice: Agree each screen as static HTML first, and test the app for parity against it. Alternative: Design in the code. Reason: A reviewer can approve a screen without reading React, and a test fails when the app drifts from it.

Region tags

Each section of a screen carries `data-region`, a slug, and `data-req`, the requirement ids it proves. Tests read them for parity, and `capture.mjs` reads them to measure this guide's red boxes.

`src/Uinsure.Policies.Web/app/shell/Rung.tsx:15`

```
export function Rung({ region, req, ordinal, heading, children }: RungProps) {
  const headingId = useId();
  return (
    <section className="s-rung" data-region={region} data-req={req} aria-labelledby={headingId}>
      <div className="s-label">
        <p className="s-ordinal">{ordinal}</p>
```



From git to the screen

How the screens are tested

Kind	Where	What it proves	Command
Unit and kit	`tests/Uinsure.Policies.Web.Tests/unit`, `tests/Uinsure.Policies.Web.Tests/kit`	Components and screens render and behave, in jsdom	`npm --prefix src/Uinsure.Policies.Web run test`
End to end	`tests/Uinsure.Policies.Web.Tests/e2e`	Each requirement against the running app and the live API	`npm --prefix src/Uinsure.Policies.Web run e2e`
Axe	`e2e/U0b/a11y-1.spec.ts`, `e2e/W1/a11y-1.spec.ts`	WCAG 2.2 AA on every route, in both themes	Part of the end-to-end run
Parity	`e2e` (`support/parity.ts`)	Each screen's regions match the signed-off static screen	Part of the end-to-end run
Visual regression	`tests/Uinsure.Policies.Web.Tests/e2e/regression`	Each screen's pixels match its baseline in `docs/ui/baselines`	Part of the end-to-end run, on Windows

Run the unit tests and the end-to-end suite

1. Run every unit and kit test once.

   ```
   npm --prefix src/Uinsure.Policies.Web run test
   ```

   You should see: All files pass. The run took 53 s (one worker per file).

   ```
   …
    RUN  v5.0.5 <repo>
    Test Files  63 passed (63)
         Tests  421 passed (421)
      Start at  17:56:42
      Duration  52.18s (tests 83%, environment 12%, import 4%, transform 1%)
       Isolate  63 workers spawned · ~1.36s startup each (spawn + environment, per file)
                at least ~3.16s faster with isolate: false — reuses workers across files instead of
   one per file
   ```

2. Run the end-to-end suite. It builds the app, starts it on 4170 and starts its own API with a fresh database. Stop any API on 5080 first.

```
npm --prefix src/Uinsure.Policies.Web run e2e
```

You should see: Exit 1 after 3646.6 s: 475 passed, 457 failed, 0 flaky.

```
Running 932 tests using 10 workers
…
  457 failed
  475 passed (1.0h)
```

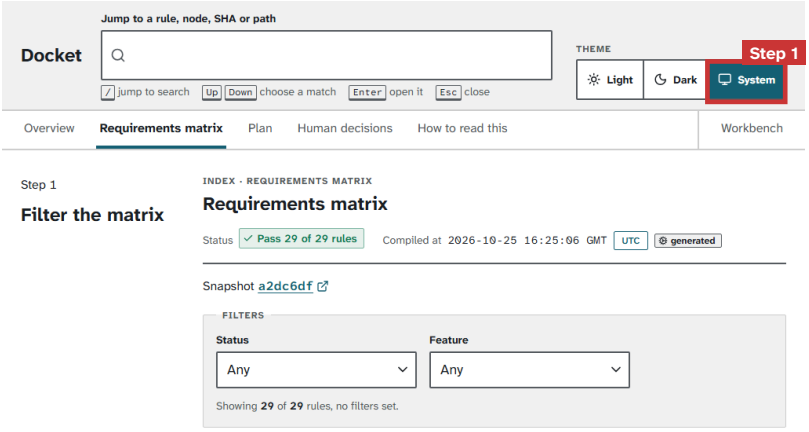
This run did not pass
On the machine that reprinted this guide the full run finished with exit 1. The failures read in its output are timeouts on the live API (a POST that was not answered within its wait, or a page that did not load), and the same specs pass when run alone: `W1/wb-3`, `N3/busy-1`, `N4/honest-1` and `W1/wb-1` all passed. The cause of the full-run failures has not been established, and nothing in this guide was changed to hide it. Run the specs of the screen you changed, and treat a full-run failure as a finding to report.

8 AJ8 · Accessibility

You decide Whether you use the keyboard or a pointer, and which theme.

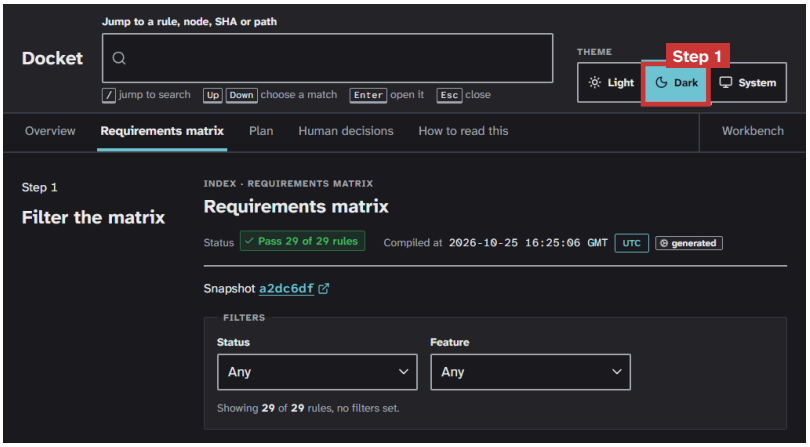
Docket gives you Keyboard paths to every control, light, dark and system themes, and an axe audit of every screen.

Key	What it does
/	Jump to the search box from anywhere on the page
Up and Down, Enter, Esc	Choose a match, open it, close the list
J and K, Tab	On rung screens: next and previous rung; Tab moves through the links in a rung



Keyboard focus on the theme control, light theme. `#/rules` Requirement tags in view: `MATRIX-1`

Step 1: The **System** button of the theme control, the chosen one. Move to the control with Tab; its focus ring shows more clearly in the dark shot below. The page's own *Step 1 Filter the matrix* is its rung label, not this one.



The same control after choosing Dark. `#/rules` Requirement tags in view: **MATRIX-1**

Step 1: The **Dark** button of the theme control, with its focus ring. Contrast holds in both themes: the audit runs in each.

What axe checks

The Playwright suites run axe at WCAG 2.2 AA on every route the app builds, in light and dark, at 1440 and 390 pixels wide. The same specs check that every control is reachable by keyboard, that focus is visible, and that a status is never shown by colour alone. The activity lanes carry a text table as their equivalent.

`tests/Uinsure.Policies.Web.Tests/e2e/U0b/support/app.ts:136`

```
export async function axeViolations(page: Page): Promise<string[]> {
  const results = await new AxeBuilder({ page }).withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa',
'wcag22aa']).analyze();
  return results.violations.map(
    (v) => `${v.id} (${v.impact ?? 'no impact'}): ${v.nodes.map((n) => n.target.join(' ')).join(',
')}`,
  );
}
```

`src/Uinsure.Policies.Web/app/shell/Frame.tsx:56`

```
export function Frame() {
  const trace = useTraceFile();
  const [theme, setTheme] = useState<ThemeChoice>('system');
  useAppliedTheme(theme);

  // PERF-1: the first screen is interactive once the trace has answered and its page is committed.
Marked once.
```

`src/Uinsure.Policies.Web/kit/patterns/search/Search.tsx:72`

```
      if (event.key !== shortcut || event.ctrlKey || event.metaKey || event.altKey) return;
      const target = event.target as HTMLElement | null;
      if (target && (target.isContentEditable || TEXT_FIELDS.has(target.tagName))) return;
      event.preventDefault();
      inputRef.current?.focus();
```

Check your understanding

Question	Answer
Is every rule evidenced?	The matrix lists every rule with its requirement, tests and commits. Where a commit is missing the page says so.
How do I check one rule?	Open its rule page: requirement, red-before-green rungs, verification recipe and links out.
How do I see the build was test-first?	A node's walkthrough lists its commits in order, tagged red, stub or green.

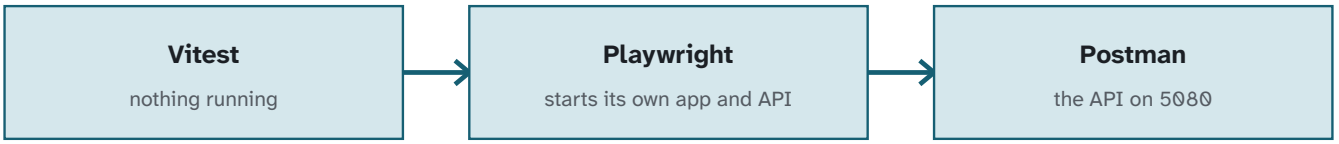
Question	Answer
How do I read a failed verdict?	Open the verdict, then its findings round and the commits whose `Fixes-Finding` trailers name each finding.

Run the tests yourself

Postman for the API's requests, Playwright for the screens, Vitest for the units.

The workbench sends the same requests the Postman collection sends. Every runner is below with its command and a real result. Run each from the directory it names. Visual Studio and `dotnet test` are in the API user guide.

▼ In depth



What each runner needs first

Postman app (MAN-1)

1. Import `tests/postman/Uinsure.Policies.postman_collection.json` and `tests/postman/local.postman_environment.json` with Import.
   The collection and the *local* environment appear in the sidebar.
2. Select the *local* environment, start the API on 5080, and open the collection's Runner.
   The Runner lists the requests in folder order.
3. Run the whole collection, one folder, or one request. Each test name starts with its rule id.
   Green ticks per request; a red cross names the failing assertion.

The folders, and the rule ids their requests prove, read from the collection:

Folder	Requests	Rule ids
F0b Walking skeleton	2	`HEALTH`
F1 Sell	39	`SELL-1` `SELL-2` `SELL-3` `SELL-4` `SELL-5` `SELL-6` `SELL-7` `SHAPE`
F1b Layered models	5	`SHAPE`
F2 Retrieve	14	`GET-1` `SHAPE` `SELL-7`
F3 Cancel	33	`CANCEL-1a` `CANCEL-1b` `CANCEL-1c` `CANCEL-2` `CANCEL-3` `CANCEL-6` `CANCEL-7` `COULD-1` `COULD-2` `CANCEL-4` `COULD-2a` `COULD-2b` `COULD-2c` `GET-1`
F4 Renew	18	`RENEW-1/2` `RENEW-3b` `RENEW-5` `COULD-3` `RENEW-6` `GET-1` `RENEW-3a` `RENEW-4`

Postman CLI (MAN-2)

1. Run one folder against the running API. No login is needed.

```
postman collection run tests/postman/Uinsure.Policies.postman_collection.json -e
tests/postman/local.postman_environment.json -i "F2 Retrieve"
```

   A summary table. The counts are requests, scripts and assertions executed and failed.

```
┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│                requests │                 25 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│            test-scripts │                 14 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│      prerequest-scripts │                 19 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│              assertions │                 26 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 1867 ms                                       │
└───────────────────────────────────────────────────────────────────┘
```

> **Provenance**
> This output was captured on the F8 run of 2026-10-23, against the same collection (1867 ms (the F2 Retrieve folder)), with the base URL overridden to the API port in use then. The Postman CLI could not start on the machine that reprinted this guide: it stopped with a Private Cloud manifest error from the machine's registry policy. No Postman app screenshot was taken.

Playwright (MAN-5)

Run these from `src/Uinsure.Policies.Web`. The runner finds the config there, and the reports land beside it.

1. Install the browser once.

```
npx playwright install chromium
```

   Chromium is downloaded to the Playwright cache.

2. Run everything. It builds the app and starts it, and starts its own API.

```
npm run e2e
```

   The same run as AJ7. Exit 1 after 3646.6 s: 475 passed, 457 failed, 0 flaky.

```
  457 failed
  475 passed (1.0h)
```

3. Run one spec, and one test by a rule id or screen name with `-g`.

```
npx playwright test --config playwright.config.ts W1/wb-1 -g "WB-1"
```

Exit `0` in 29.4 s.

```
…
  ✓  18 ..\..\tests\Uinsure.Policies.Web.Tests\e2e\W1\wb-1.spec.ts:206:7 › parity › WB-1 · parity: the sell form has its reference's title, shell and navigation (899ms)
  ✓  19 ..\..\tests\Uinsure.Policies.Web.Tests\e2e\W1\wb-1.spec.ts:216:9 › parity › WB-1 · parity: the sell form's policy-fields-reference-type-start-date-amount-payment-method-auto-renew region carries its reference's components (856ms)
  ✓  20 ..\..\tests\Uinsure.Policies.Web.Tests\e2e\W1\wb-1.spec.ts:216:9 › parity › WB-1 · parity: the sell form's reference-as-stored-normalisation-preview region carries its reference's components (796ms)
  ✓  21 ..\..\tests\Uinsure.Policies.Web.Tests\e2e\W1\wb-1.spec.ts:216:9 › parity › WB-1 · parity: the sell form's policyholders-1-to-3 region carries its reference's components (725ms)
  ✓  22 ..\..\tests\Uinsure.Policies.Web.Tests\e2e\W1\wb-1.spec.ts:216:9 › parity › WB-1 · parity: the sell form's property region carries its reference's components (773ms)
  ✓  23 ..\..\tests\Uinsure.Policies.Web.Tests\e2e\W1\wb-1.spec.ts:221:7 › parity › WB-1 · parity: the sell form colours only from tokens, light and dark (1.0s)

  23 passed (28.0s)
```

4. Watch it run: UI mode, a visible browser, or the step-by-step debugger.

```
npx playwright test --ui
npx playwright test --headed
npx playwright test --debug
```

Not run for this guide (interactive).

5. Write an HTML report with traces, then open it. In the report, open a test and choose its trace to step through it.

```
npx playwright test --reporter=html --trace on
```

Not run for this guide. Then, as its own command:

6. Open the report.

```
npx playwright show-report
```

The report opens in the browser.

7. Run the visual-regression suite on its own.

```
npx playwright test regression/
```

Each screen's pixels are compared with its baseline in `docs/ui/baselines`.

> **Update baselines only on purpose**
> The baselines are Windows renders and are signed off with the screens, so the regression folder is skipped on Linux. Update them only deliberately, with `npx playwright test regression/ --update-snapshots`, after a signed-off design change. Never update one to make a failure go away.

Vitest (MAN-6)

1. Run every unit and kit test once.

```
npm --prefix src/Uinsure.Policies.Web run test
```

All files pass in 53 s.

```
…
 Test Files  63 passed (63)
      Tests  421 passed (421)
   Start at  17:56:42
   Duration  52.18s (tests 83%, environment 12%, import 4%, transform 1%)
    Isolate  63 workers spawned · ~1.36s startup each (spawn + environment, per file)
             at least ~3.16s faster with isolate: false — reuses workers across files instead of one per
file
```

2. Run one file: a name fragment filters the files. Run from `src/Uinsure.Policies.Web`.

```
npx vitest run --config vite.config.ts wb-7-requests-drawer
```

Exit `0` in 3.6 s.

```
 RUN  v5.0.5 <repo>
 Test Files  1 passed (1)
      Tests  3 passed (3)
   Start at  17:57:35
   Duration  2.30s (environment 49%, import 24%, transform 19%, tests 8%)
```

3. Watch mode: it reruns the tests that a file change touches. Press `q` to quit.

```
npx vitest --config vite.config.ts
```

Not run for this guide (interactive).

Every runner (MAN-7)

Runner	What it tests	Use when	Command or click	Needs running first	Time
Postman app	The API's requests, by rule id	Exploring the API by hand	Collection Runner	The API on `5080`	Shown by the Runner itself
Postman CLI	The same requests	Scripted checks	`postman collection run …`	The API on `5080`	1867 ms (the F2 Retrieve folder)
Playwright, all	Screens, axe, parity, regression	Before a UI change lands	`npm run e2e`	Nothing: it starts its own	3646.6 s
Playwright, one spec	One requirement's specs	While fixing one screen	`npx playwright test W1/wb-1 -g "WB-1"`	Nothing: it starts its own	29.4 s
Vitest, all	Units and the kit	While coding	`npm run test`	Nothing	53 s
Vitest, one file	One file's tests	While coding one component	`npx vitest run … <name>`	Nothing	3.6 s

Check your understanding

Question	Answer
Which runner needs the API already running?	Postman, the app or the CLI. Playwright starts its own API, so stop any API on `5080` first.
How do I run just one screen's test?	`npx playwright test regression/matrix` for its baseline, or `-g` with its rule id for its specs.

Question	Answer
When may I update a visual baseline?	Only deliberately, with `--update-snapshots`, after a signed-off design change.
Where are the .NET suites?	In the API user guide: Visual Studio's Test Explorer and `dotnet test`.

Troubleshooting

Most problems are a missing API, a missing trace or a missing browser.

Each row is a problem met while building Docket or this guide.

1

TJ1 · Troubleshooting

> **You decide** Which symptom you have.

> **Docket gives you** The cause and the fix, from the problems met while building Docket and this guide: the trail's findings, `docs/lessons.md` and the capture runs.

Symptom	Cause	Fix
The workbench says *The API did not answer*	The API is not running, or `DOCKET_API_PORT` names the wrong port	Start the API on its http profile; set `DOCKET_API_PORT` to its port and restart the preview
Explorer screens say the data is not available	`trace.json` was not compiled	`npm --prefix src/Uinsure.Policies.Web run trace`, then rebuild
On the dev server the explorer says the trail is compiling	The server answers 503 *compiling* until the compile ends (lessons, bug-002/N1)	Wait for Vite's *Local:* line, then reload
`Packages are not restored` when starting the dev server	A fresh clone has no `node_modules`	`npm --prefix src/Uinsure.Policies.Web ci`, then start again
Port 4170 is in use	The dev server uses `--strictPort`	Stop the other process, or pass another `--port`
`DOCKET_API_PORT=5080 npm …` is not recognised	PowerShell does not take a variable prefix	Use Git Bash, or set `$env:DOCKET_API_PORT` first
The end-to-end run stops before any test with a setup fault	An API that this run did not start is already answering on the port	Stop the API on 5080, then run again
The full end-to-end run fails many tests with *POST /api/policies is answered* or a 90 s timeout	The tests wait on the live API and it did not answer in time. Seen on the reprint machine; the cause is not established, and the same specs pass alone	Run the specs of the screen you changed (`W1/wb-3`, `N3/busy-1`) and report the full-run failure as a finding
Playwright says the browser is missing	Chromium is not installed	`npx playwright install chromium`
Visual regression fails on Linux or macOS	The baselines are Windows renders; Linux skips the folder	Run on Windows; update baselines only deliberately
A rule's count differs between the matrix and its rule page	The matrix counts the tests at the node that built the rule; the rule page lists every test traited with it	Expected: the matrix's own *Step 3* explains it
A rule page says no green commit is recorded	The trail lists no green commit for that rule; the page does not invent one	Read the red commits, the QA step and the sign-off; this is a gap in the trail, not in the page
Renew answers 422 on a new policy	RENEW-1 and RENEW-2: the window opens 30 days before the end date	Expected; renew inside the window
A sell answers 409	SELL-1: the reference already exists, in any letter case	Use a new reference
The request log is empty after a reload	It lists the requests since the page opened	Send the request again
The Postman CLI stops with a Private Cloud manifest error	A registry policy on the machine names a manifest the CLI cannot read	Ask whoever manages the machine, or use the Postman app

Check your understanding

Question	Answer
The workbench says no answer. What do I check first?	That the API is running on the port `DOCKET_API_PORT` names.
Why does the end-to-end run refuse to start?	An API it did not start is already on the port, so its database cannot be known to be fresh.
Is a rule page with no green commit a defect in Docket?	No. It reports a gap in the trail, and says so.